

I.P.(P.G.) COLLEGE, CAMPUS-2, BULANDSHAHR



A

PROJECT REPORT

Entitled

“CampVerse”

As

Major Project

Submitted in the partial fulfillment the degree of

Bachelor of Computer Application

Batch (2023-2026)

CH. CHARAN SINGH UNIVERSITY, MEERUT

Under The Distinguish Guidance of:-

Guide Name: -

Miss Rifa Rana (Asst.Prof.)

Dept. Of Computer Science

I.P.(P.G) College Campus-II,

Bulandshahr,

Submitted By:

Group No:- G -65

Name: Kamna Singh

Roll no.: [230955010510]

ACKNOWLEDGEMENT

We take this opportunity to express our sincere thanks and deep gratitude to all those people who extended their whole hearted co-operation and have helped us in completing this project successfully.

We acknowledge the effort of those who have contributed significantly to our project. We express our sincere attitude and thank-fullness towards **Dr. T.N. MISHRA** (Principal of I.P.(P.G.) College Campus-2 NH-91, Delhi Road, Bulandshahr) has provided such a talented faculty to us then We would like to thank **Dr. Sanjay Kumar** (Head of the Department) their guidance throughout BCA.

We sincerely thank to our guide **“Miss. Rifa Rana”** for her inspiring guidance and constant motivation. Every time we were lurking in the dark, she showed us the way out.

We would like to express our gratitude to other faculty members of our college for their reviews and many helpful comments.

Finally, the project would not have been possible without confidence, endurance and support of our group members.

Name of Group Member

Kamna Singh [230955010510]

STUDENT DECLARATION

We/I, solemnly affirm and declare that the project “CampVerse” under the guidance of “Rifa Rana” our original work. No part of this work whether documentation and coding has not been copied or taken in any form or by any means.

We/I certify that our/my work is original and it has been made as Major Project based on winter training held after BCA V semester.

Group member's name with Roll no.

Signature

1. Kamna Singh [230955010510]

CERTIFICATE

This is to certify that the project report entitled, “CampVerse” is a benefited work done by Kamna Singh (230955010510). Students of BCA V sem. (Batch 2023-2026) as major project report carried out under my supervision and guidance.

To the best of my knowledge this project work is genuine.

.....

Signature of Guide

Guide Name:-

Miss Rifa Rana (Asst. Prof.)

Dept. of Computer Science

I.P.(P.G.) College, Campus-2, Bulandshahr

Date :

TABLE OF CONTENTS

Topic

1. About The Project
 - i. Introduction
 - ii. Objective
2. Preliminary Investigation
 - i. Tools/Platform used
 - ii. Hardware/Software Requirement
3. Feasibility Study
 - i. Operational Feasibility
 - ii. Technical Feasibility
 - iii. Economic Feasibility
4. Information Gathering (Mention tools used for Information Gathering)
 - i. On Site Observation
 - ii. Questionnaire
 - iii. Interview
5. Requirement Analysis & Specification (Final Conclusion About the Information Gathering)
 - i. Data Flow Diagram(DFD)
6. Logical Design
 - i. Entity Relationship Diagram(ERD)
 - ii. Data Dictionary
 - iii. Database/Table Design
 - iv. PERT/GANTT Chart
(as per the project requirement)
7. Physical Design
 - i. Output Requirement
 - User Interface Design
 - Output Report Design
 - ii. Input Requirement
8. Process Design
 - i. Coding
 - ii. Constraints/Check
9. Testing
10. Maintenance
11. Screen Shots
 - i. Forms
 - ii. Reports
12. Limitations/Future Scope
13. References/Bibliography

ABOUT THE PROJECT

INTRODUCTION

In the modern educational landscape, campuses face numerous challenges in efficiently managing student activities, communication, and administrative tasks. Manual processes, scattered information, and limited accessibility often result in delays, miscommunication, and inconvenience for both students and faculty. To address these issues, our project CampVerse has been developed as a comprehensive, web-based platform that integrates multiple campus-related services into a single digital ecosystem. This platform is designed to improve operational efficiency, enhance student engagement, and create a connected campus environment.

CampVerse is a multi-module system that brings together five essential functionalities: Lost and Found Management, Digital Notice Board, Second-Hand Goods Marketplace, Student Feedback System, and College Management. Each module is specifically designed to address a particular aspect of campus life, making it easier for students, faculty, and administrators to interact effectively and access information quickly.

1. Lost and Found Management System

The Lost and Found module is designed to tackle one of the most common campus problems—misplaced or unclaimed belongings. Traditionally, lost items are announced verbally or kept in administrative offices without proper tracking. CampVerse replaces this outdated method with a modern, organized, online system.

Features of This Module:

- Students can report lost items with details and images.
- Anyone who finds an item can upload details of found items.
- The system helps match lost items with found items quickly.
- All reports are stored in a centralized digital database.
- Reduces confusion, improves transparency, and increases recovery rates.

2. Digital Notice Board

A digital notice board is essential for modern campuses where students expect instant information access. CampVerse offers a dynamic, real-time notice board that ensures students can stay updated at all times.

Features of This Module:

- Displays important college announcements.
- Updates exam schedules, holidays, events, and circulars instantly.
- Accessible from any device, eliminating the need to visit physical boards.
- Ensures no student misses critical information.
- Reduces dependency on printed notices and classroom announcements.

This module makes communication fast, accurate, and dependable.

3. Second-Hand Goods Marketplace

College students often need affordable books, materials, and essential items. On the other hand, students leaving the campus want to sell items they no longer need. CampVerse provides a safe campus-only marketplace that encourages responsible buying and selling.

Features of This Module:

- Students can buy/sell used books, calculators, laptops, or hostel items.
- Only college-verified users can use the platform, ensuring safety.
- Helps reduce financial burden on students.
- Promotes sustainability by encouraging reuse of goods.
- Offers a trusted, organized alternative to random social media groups.

This marketplace strengthens student connections while providing value and safety.

4. College Management System

This module acts as the central digital guidebook of the entire college. It organizes all institutional information in one accessible location so students, faculty, and visitors no longer need to search through multiple sources.

Features of This Module:

- Complete list of departments with descriptions.
- Faculty details including qualifications and areas of expertise.
- Course details, syllabus overview, and eligibility information.
- Information about campus facilities such as library, labs, sports, hostel, etc.
- Academic resources like the academic calendar, exam schedules, and rules.

This module ensures that students can access accurate information anytime without confusion.

5. Student Feedback System

Feedback is an essential tool for improving the academic environment. CampVerse integrates a modern feedback system that encourages students to express their opinions in a structured and comfortable manner.

Features of This Module:

- Students can share feedback about teachers, courses, and campus facilities.
- Helps administration identify strengths and weaknesses.
- Encourages honest and constructive responses.
- Ensures systematic storage and analysis of feedback.
- Supports decision-making and improves overall campus quality.

Objectives of CampVerse

The primary aim of CampVerse is to create a comprehensive, integrated, and user-friendly digital platform that simplifies campus life for students, faculty, and administration. Modern educational institutions face multiple challenges, including inefficient communication, lack of organized information, difficulty in reporting and recovering lost items, absence of secure marketplaces for student goods, and limited mechanisms for collecting student feedback. Traditional manual systems are often slow, confusing, and error-prone, which affects both student experience and administrative efficiency.

CampVerse addresses these challenges by bringing together five key modules:

1. Lost and Found Management System
2. Digital Notice Board
3. Second-Hand Goods Marketplace
4. College Management System
5. Student Feedback System

The following detailed objectives explain the purpose and goals of CampVerse and the impact it aims to achieve within the campus ecosystem.

1. Centralization of Campus Services

One of the most important objectives of CampVerse is to centralize all essential campus services into a single, unified platform. Traditionally, students must rely on different tools or manual systems to complete simple tasks: checking notices on a physical board, reporting a lost item in the office, asking about course details in the department, or using informal social media groups to sell items. This scattered approach is inefficient and time-consuming.

2. Enhancing Communication Between Students and Administration

Effective communication is the backbone of any educational institution. CampVerse's Digital Notice Board and integrated college information module aim to bridge the communication gap between students and the administration.

- Students can instantly receive updates about exams, holidays, events, and announcements without needing to visit physical notice boards.
- The platform ensures that all students receive the same information simultaneously, preventing confusion or miscommunication.

3. Reducing Manual Workload and Administrative Effort

Manual systems such as paper-based registers, offline feedback forms, and verbal announcements are time-consuming and prone to errors. One of the main objectives of CampVerse is to automate and digitalize these processes.

- Lost and Found items can be reported online, reducing the need for manual entries.
- Feedback collection becomes systematic and digital, allowing for faster analysis.
- Notices and academic updates can be posted in seconds instead of physically printing them.

4. Providing a Safe and Transparent Lost and Found System

Students frequently lose personal belongings like books, gadgets, or ID cards on campus. The objective of CampVerse's Lost and Found Management System is to create a secure, transparent, and efficient way to report, track, and recover lost items.

- Users can upload details and images of lost or found items.
- The system matches lost items with found items automatically, increasing recovery rates.
- Complete records are stored digitally, reducing disputes and confusion.

5. Promoting Sustainability and Affordability Through a Marketplace

College students often have limited budgets but a high demand for books, gadgets, and other essentials. CampVerse includes a Second-Hand Goods Marketplace to address this challenge.

- Students can sell items they no longer need and buy items at lower costs.
- Campus-only access ensures safety and authenticity.
- This objective promotes sustainability by encouraging recycling and reuse of items within the campus.

6. Delivering Accurate and Structured College Information

Accessing structured, accurate information about departments, courses, faculty, and facilities is often difficult for students. CampVerse's College Management System addresses this by providing a digital repository of all institutional information.

- Students can explore department details, faculty profiles, course syllabi, and campus facilities without visiting multiple offices.
- New students can adapt quickly by having all relevant information in one place.

7. Fostering a Feedback-Driven Environment

Feedback is a critical tool for improving teaching quality, course content, and campus facilities. CampVerse's Student Feedback System ensures that students can voice their opinions conveniently and securely.

- Honest feedback helps faculty and administration identify strengths and weaknesses.
- Feedback data is stored systematically for analysis, enabling informed decision-making.
- The platform encourages students to participate in campus development initiatives.

PRELIMINARY INVESTIGATION

TOOLS AND PLATFORM USED

The development of CampVerse, a comprehensive campus management system, involves a combination of advanced tools, programming languages, frameworks, and platforms. These tools have been carefully chosen to ensure that the system is efficient, scalable, secure, and user-friendly, enabling smooth interaction between students, faculty, and administrators. The following sections detail the tools and platforms used in the project and their relevance to various modules.

1. HTML5 (HyperText Markup Language)

- Purpose: To create the structure and content of all web pages.
- Usage: Layouts for all modules (Lost & Found, Notice Board, Marketplace, College Info, Feedback), forms, tables, buttons, and images.
- Advantage: Provides semantic elements for readability, accessibility, and browser compatibility.

2. CSS3 (Cascading Style Sheets)

- Purpose: To style and format HTML content.
- Usage: Designing layouts, coloring, fonts, buttons, tables, hover effects, and responsive designs.
- Advantage: Makes the platform visually appealing and consistent across devices.

3. JavaScript (JS)

- Purpose: To add interactivity and dynamic behavior.
- Usage: Form validation, filtering marketplace items, dynamically updating notices, and creating interactive modals/pop-ups.
- Advantage: Enables real-time interactivity without a backend.

4. Bootstrap (CSS Framework)

- Purpose: To make the platform responsive and mobile-friendly.
- Usage: Navigation bars, grids, cards, buttons, and responsive layouts.
- Advantage: Saves development time and ensures consistent design across all devices.

5. React (Frontend Framework)

- Purpose: React is used to build fast, dynamic, and component-based user interfaces for web applications.
- Usage: React helps developers create reusable UI components and efficiently update the user interface based on data changes.
- Advantages: React provides high performance, reusable components, a virtual DOM for fast rendering.

5. Icons and Fonts

- Tools: Font Awesome (icons), Google Fonts (custom fonts)
- Purpose: To enhance visual appeal and user experience.
- Usage: Icons for buttons, navigation, alerts; stylish fonts for headings and content.
- Advantage: Improves usability and interface aesthetics.

6. Text/Code Editor

- Tools: VS Code, Sublime Text
- Purpose: To write, edit, and organize frontend code.
- Usage: Coding HTML, CSS, and JavaScript ,React for all modules.
- Advantage: Provides syntax highlighting, live preview, and debugging features.

7. Web Browser

- Tools: Google Chrome, Mozilla Firefox, Microsoft Edge
- Purpose: To test and run the frontend application.
- Usage: Checking responsiveness, functionality, and debugging using developer tools.
- Advantage: Ensures cross-browser compatibility and proper rendering.

The **frontend-only version of CampVerse** uses the above tools and platforms to create a **fully functional, interactive, and user-friendly campus management system**. All five modules—Lost & Found, Digital Notice Board, Second-Hand Marketplace, College Management, and Student Feedback—work efficiently using only frontend technologies, making the project **easy to demonstrate, visually appealing, and scalable for future backend integration**.

HARDWARE AND SOFTWARE REQUIREMENT

1. Hardware Requirements

Since CampVerse is a frontend-only web application, it does not require high-end hardware. The following specifications are sufficient to develop, run, and test the platform efficiently:

Component	Minimum Requirement	Recommended Requirement
Processor (CPU)	Intel Core i3 or equivalent	Intel Core i5 or above
RAM	4 GB	8 GB or more
Storage (HDD/SSD)	100 GB free space	256 GB SSD for faster performance
Display	1024 x 768 resolution	1366 x 768 or higher resolution
Input Devices	Keyboard, Mouse	Keyboard, Mouse, Touchpad optional
Network	Internet connection for library references & updates	High-speed Internet connection for faster resources and testing

Note: Since the platform is frontend-only, it can run smoothly on low to mid-range computers, and can also be tested on laptops, desktops, or tablets.

2. Software Requirements

CampVerse frontend development requires standard web development tools and platforms:

Category	Software / Platform	Purpose
Operating System	Windows 10/11, Linux, or macOS	To run development tools and web browsers
Web Browser	Google Chrome, Mozilla Firefox, Microsoft Edge	To test and run the frontend application, check responsiveness, debug using developer tools
Code Editor / IDE	VS Code, Sublime Text, Atom	To write HTML, CSS, JavaScript code
Frontend Languages	HTML5, CSS3, JavaScript	Core languages for designing and developing the platform
CSS Framework and Frontend Framework	Bootstrap and React	For responsive layouts, prebuilt components, and mobile-friendly design
Icons & Fonts	Font Awesome, Google Fonts	To enhance UI aesthetics and usability
Local Storage	Browser's local storage	To store temporary data such as lost items, feedback, or marketplace entries
Image Editor (Optional)	Adobe Photoshop / Canva / GIMP	To create images, icons, or UI elements for the project

FEASIBILITY STUDY

OPERATION FEASIBILITY

Operation Feasibility refers to the extent to which a system or software can be operated efficiently and effectively by its intended users. It evaluates whether the system is **user-friendly, easy to use, and practical** for its intended environment. For CampVerse, operation feasibility focuses on ensuring that **students, faculty, and administrators** can effectively use the platform to manage campus activities without difficulty.

1. User-Friendliness

- CampVerse has been designed with a **simple and intuitive interface**, allowing users to navigate easily between modules.
- All five modules—Lost & Found Management, Digital Notice Board, Second-Hand Marketplace, College Management System, and Student Feedback System—are **clearly labeled and structured**, reducing confusion for first-time users.
- Forms and interactive components are **easy to fill and submit**, with real-time feedback through alerts and notifications.
- Minimal training is required for students or staff to operate the system effectively.

2. Accessibility

- CampVerse is a **web-based platform**, accessible from any device with a web browser, including **desktop computers, laptops, tablets, and mobile devices**.
- Users do not need to install any heavy software or backend tools.
- The responsive design (powered by **Bootstrap**) ensures that the platform adapts to different screen sizes for smooth operation.

Impact: Students and faculty can access campus information anytime, anywhere, which increases overall usability.

3. Ease of Data Handling

- Although the backend is removed, CampVerse uses **local storage** in the browser to temporarily store user data, such as:
 - Lost and found items
 - Student feedback
 - Marketplace listings
- This allows users to **enter, view, and manage data locally** without relying on servers.
- Interactive JavaScript functionality ensures real-time updates, such as filtering marketplace items or displaying new notices instantly.

4. Error Handling and Reliability

- Form validation in all modules prevents incorrect data submission.
- Alerts and notifications inform users when an action is completed successfully or if there is an error.

- Users can re-enter data or update entries easily without losing information.
- Even without a backend, the system ensures **a reliable and consistent user experience** within the browser session.

5. Maintenance and Support Feasibility

- CampVerse is **easy to maintain** since all code is on the client-side.
- Any updates or bug fixes can be implemented directly in HTML, CSS, or JavaScript files.
- No server maintenance or database management is required in the frontend-only version, reducing operational complexity.
- Future improvements, such as adding new modules or integrating backend support, can be implemented incrementally.

6. Scalability for Users

- The system is **capable of handling multiple users** accessing the platform from their own devices, as each browser session maintains separate data in local storage.
- While the frontend version does not allow centralized data storage, it **demonstrates all functionalities clearly** for academic purposes.
- Operation feasibility is high for small-to-medium scale campus use or project demonstration.

7. Advantages of High Operational Feasibility

1. **Quick Learning Curve:** Students and faculty can start using CampVerse without extensive training.
2. **Instant Access:** Campus information, notices, lost items, marketplace listings, and feedback forms are accessible immediately.
3. **Minimal Hardware Requirements:** The platform works smoothly on standard laptops or desktops.
4. **Low Operational Cost:** No need for server hosting, database management, or backend infrastructure.
5. **Ease of Updates:** Simple code updates can enhance functionality without affecting users.

TECHNICAL FEASIBILITY

Technical Feasibility evaluates whether the system can be developed and implemented using the available technical resources, including hardware, software, and development tools. It assesses whether the project can be **built efficiently, maintained easily, and operated smoothly** using existing technology.

For **CampVerse**, technical feasibility focuses on **frontend-only development** using modern web technologies, ensuring that all five modules—**Lost & Found Management, Digital Notice Board, Second-Hand Marketplace, College Management System, and Student Feedback System**—function effectively without a backend.

1. Available Technology

CampVerse leverages widely available and proven **frontend technologies**, which makes development and deployment technically feasible:

- **HTML5:** Provides the structure for all modules and pages.
- **CSS3:** Styles the interface with responsiveness and visual appeal.
- **JavaScript:** Adds interactivity, validation, and dynamic behavior.
- **Bootstrap:** Offers responsive layouts, prebuilt components, and cross-device compatibility.
- **React :** React is used to build fast, dynamic, and component-based user interfaces for web applications.
- **Local Storage:** Enables temporary data persistence without a server or database.
- **Optional Libraries:** jQuery, Swiper.js, and Chart.js enhance functionality (sliders, charts, dynamic content).

2. Hardware and System Requirements

The system is designed to run on **standard hardware**, minimizing technical limitations:

Component	Requirement
Processor	Intel Core i3 or higher
RAM	4 GB minimum (8 GB recommended)
Storage	100 GB free space (256 GB SSD recommended)
Display	1024x768 resolution minimum
Input	Keyboard, Mouse
Network	Internet connection (for development and optional online libraries)

3. Software Requirements

The frontend-only approach relies on software that is **readily available and widely used**, including:

- **Operating Systems:** Windows, Linux.
- **Code Editors/IDE:** Visual Studio Code, Sublime Text, Atom
- **Web Browsers:** Chrome, Firefox, Edge.

- **Design Tools:** Canva, Adobe Photoshop (optional for icons and UI)

4. Development Complexity

- CampVerse frontend development is **moderately complex**:
 - HTML5 handles structure
 - CSS3 and Bootstrap handle design and responsiveness
 - JavaScript handles logic, validation, and interactivity
 - React handles the creation and management of the user interface (UI) of a web application.
- No backend means no need to manage databases, server logic, or APIs.
- Local storage can temporarily manage module data, making the system functional for **demonstration and small-scale use**.

5. Scalability and Future Expansion

- The system is designed to be **scalable**:
 - Additional modules (attendance, online exams, or online payments) can be added in the future.
 - Backend integration can be implemented later for permanent data storage.
 - The current frontend structure ensures easy expansion of UI and features without major redesign.

6. Reliability and Maintainability

- **Reliability:** Frontend modules are tested on modern browsers and devices, ensuring consistent operation.
- **Maintainability:** Code is modular and organized, making future updates, bug fixes, and enhancements easy.
- **Minimal Downtime:** No server or database dependencies reduce risks of operational failure.

7. Limitations and Considerations

- Data is stored only in **browser local storage**, so permanent data persistence is not possible.
- Multi-user centralized data management is not available in the frontend-only version.
- Advanced analytics, real-time notifications across devices, and remote data access require backend integration.

ECONOMIC FEASIBILITY

Economic Feasibility **evaluates whether the proposed system is cost-effective and financially viable for development, implementation, and maintenance. It focuses on analyzing the costs involved in the project and determining whether the benefits outweigh the expenditures. For CampVerse, the frontend-only approach ensures low-cost development and implementation while providing maximum utility to students and faculty.**

1. Cost of Hardware

Since CampVerse is a **frontend-only application**, it can run on standard computers and does not require specialized or high-end hardware.

Hardware Component	Estimated Cost	Remarks
Laptop/Desktop Computer	₹25,000 – ₹50,000	Standard specification for development and testing
Internet Connection	₹500 – ₹1,000 per month	Required for development and optional online resources
Optional Accessories	₹500 – ₹2,000	Keyboard, Mouse, or Touchpad for ease of use

2. Cost of Software

CampVerse uses **free or open-source software** for frontend development, which reduces expenses significantly:

Software / Platform	Cost	Remarks
HTML5, CSS3, JavaScript	Free	Core web technologies
Bootstrap , React	Free	CSS framework for responsive design
VS Code / Sublime Text	Free	Code editors/IDEs for development
Web Browsers	Free	Chrome, Firefox, Edge for testing
Font Awesome / Google Fonts	Free	Icons and fonts to enhance UI
Optional Libraries	Free	jQuery, Swiper.js, Chart.js for interactivity
Image Editing Tools	Free / Optional	Canva or GIMP for icons/UI graphics

3. Development Cost

Since the project is a **student academic project**, development costs are minimal:

- Only the **time of the developers/students** is required.
- No need to purchase backend servers or databases.
- All tools are available for free or as open-source.

4. Operational Cost

Operation of the frontend-only CampVerse system involves minimal costs:

- **Internet:** Required for optional online resources or library references.

- **Power/Electricity:** Standard electricity usage for computers.
- **Maintenance:** Simple code maintenance and updates; no server or database costs.

5. Cost-Benefit Analysis

Benefits	Cost
Centralized campus services (Lost & Found, Notice Board, Marketplace, College Info, Feedback)	Minimal hardware and software cost
Reduced administrative workload	No additional software licenses needed
Real-time accessibility and interaction	No server or database required
Easy maintenance and scalability	Only minor costs for future optional upgrades
Learning experience for students	Free educational resources and tools

6. Financial Feasibility Summary

- **Low initial investment:** Uses existing computers and free software.
- **Minimal running costs:** Only electricity and internet usage.
- **No recurring license or server costs:** Entirely frontend-based.
- **High utility and educational value:** Provides multiple campus services in one system.

The **economic feasibility of CampVerse is high**, as it requires **minimal financial resources** while providing significant functional and educational benefits. The project is suitable for academic purposes and can be implemented without creating financial strain on students or institutions.

INFORMATION GATHERING (Mention tools used for Information Gathering)

ON SITE OBSERVATION

On-Site Observation involves studying the current environment of the campus and understanding the existing processes, challenges, and needs that the proposed system, CampVerse, aims to address. This observation is essential to ensure that the system is **practical, relevant, and user-friendly** for students, faculty, and administrative staff.

1. Purpose of On-Site Observation

The main goals of on-site observation for CampVerse were:

1. To **identify problems** in the existing manual campus systems such as lost and found items, notice dissemination, student feedback, and college management.
2. To **understand user behavior** and how students and staff interact with current systems.
3. To gather data to **design a practical and effective frontend-based solution** that can be easily operated without a backend.
4. To assess the **infrastructure and resources** available for implementing the system.

2. Method of Observation

- **Site Visited:** College campus, including classrooms, administrative offices, notice boards, and student common areas.
- **Tools Used:** Notebook, pen, camera (optional), and personal interviews with staff and students.
- **Focus Areas:**
 1. **Lost and Found System** – Checking how students report lost items and retrieve them.
 2. **Notice Board Management** – Observing how notices are published and accessed by students.
 3. **Second-Hand Marketplace** – Evaluating the methods students use to buy and sell used items.
 4. **College Information** – Reviewing how students access details about courses, faculty, and departments.
 5. **Student Feedback System** – Understanding the current methods of collecting and analyzing feedback.

3. Findings from Observation

3.1 Lost and Found System

- Currently, students report lost items manually at the administrative office.
- Records are maintained on paper, often causing delays in retrieval.
- Students face difficulty in tracking lost items efficiently.

Observation: There is a clear need for a **digital, easily accessible Lost & Found Management System**.

3.2 Notice Board Management

- Notices are displayed physically on notice boards in classrooms and corridors.
- Many students miss important notices due to irregular visits or overcrowding.
- Printing notices consumes time and resources.

Observation: A **Digital Notice Board** accessible from devices would ensure timely updates and reduce manual effort.

3.3 Second-Hand Marketplace

- Students use informal groups (social media or word-of-mouth) to buy/sell items.
- No organized platform exists within the campus, leading to confusion and lack of safety.

Observation: A **campus-specific marketplace** would allow safe and convenient buying and selling of items.

3.4 College Management Information

- Students access information by visiting departments or asking peers.
- Lack of centralized information leads to confusion regarding faculty, courses, and schedules.

Observation: A **frontend-based College Management System** can consolidate all information in one accessible location.

3.5 Student Feedback System

- Feedback is collected using paper forms or scattered online links.
- Anonymity and organization of feedback are limited.
- Administration takes time to analyze and act on feedback.

Observation: A **digital, organized feedback system** would encourage participation and simplify analysis.

4. Operational Environment

- **Devices Available:** Laptops, desktops, and mobile devices accessible to students and staff.
 - **Internet Availability:** Moderate, sufficient for accessing frontend-only applications.
 - **User Skill Level:** Most students and staff are familiar with web interfaces and basic computing.
- Observation:** The existing environment is technically ready for **frontend-based web applications**, making CampVerse feasible for deployment.

QUESTIONNAIRE

The purpose of this questionnaire is to **collect information about the current campus systems**—such as Lost & Found management, notice dissemination, second-hand marketplace, college information, and student feedback—and to evaluate the **need and usability** of the proposed CampVerse platform.

Target Respondents:

- Students (Undergraduate and Postgraduate)
- Faculty members
- Administrative staff

Mode of Collection:

- Paper-based forms or
- Online forms (Google Forms, Microsoft Forms)

A. General Information

1. Name (optional): _____
2. Age: _____
3. Gender: ☐ Male ☐ Female ☐ Other
4. Role: ☐ Student ☐ Faculty ☐ Staff
5. Department / Year (if student): _____

B. Lost & Found System

1. Have you ever lost an item on campus?
☐ Yes ☐ No
 2. How easy is it to report and retrieve lost items in the current system?
☐ Very Easy ☐ Easy ☐ Difficult ☐ Very Difficult
 3. Do you face delays in finding lost items?
☐ Always ☐ Sometimes ☐ Rarely ☐ Never
 4. Would you prefer a **digital Lost & Found system** accessible via computer or mobile?
☐ Yes ☐ No
 5. Any suggestions to improve the lost and found process?
-

C. Digital Notice Board

1. How often do you check the physical notice boards on campus?
☐ Daily ☐ Weekly ☐ Occasionally ☐ Rarely
2. Do you often miss important notices?
☐ Yes ☐ No
3. Would you find it useful to have a **digital notice board** with real-time updates?
☐ Yes ☐ No

4. Which features would you like on a digital notice board? (Select all that apply)

- ☐ Alerts / Notifications
☐ Search by category
☐ Downloadable notices
☐ Others: _____

D. Second-Hand Marketplace

- Have you ever bought or sold second-hand items on campus?
☐ Yes ☐ No
- How do you usually buy/sell items currently?
☐ Word-of-mouth ☐ Social media groups ☐ Notice boards ☐ Other: _____
- How safe and organized do you find the current process?
☐ Very Safe ☐ Safe ☐ Unsafe ☐ Very Unsafe
- Would a **campus-specific online marketplace** be helpful?
☐ Yes ☐ No

E. College Management System

- How easy is it to access college information (faculty, courses, schedules)?
☐ Very Easy ☐ Easy ☐ Difficult ☐ Very Difficult
- Do you face delays or confusion in getting official information?
☐ Always ☐ Sometimes ☐ Rarely ☐ Never
- Would you like an **online College Management system** for quick access to information?
☐ Yes ☐ No

F. Student Feedback System

- How is student feedback collected currently?
☐ Paper forms ☐ Online links ☐ Verbally ☐ Not collected
 - Do you feel comfortable giving feedback in the current system?
☐ Yes ☐ No
 - Would you prefer a **digital feedback system** that is easy, anonymous, and accessible online?
☐ Yes ☐ No
 - What improvements would you suggest for the feedback system?
-

G. Overall Assessment

- How interested are you in using **CampVerse**, a platform combining all campus services digitally?
☐ Very Interested ☐ Interested ☐ Neutral ☐ Not Interested
- Which feature do you think is most useful?
☐ Lost & Found ☐ Digital Notice Board ☐ Second-Hand Marketplace ☐ College Info ☐ Feedback System
- Any other suggestions or comments about CampVerse?

INTERVIEW

The purpose of conducting interviews is to **collect detailed information and opinions** from students, faculty, and administrative staff regarding current campus systems, their challenges, and expectations from a digital platform like CampVerse.

Target Respondents:

- Students (UG and PG)
- Faculty members
- Administrative staff

Mode of Interview:

- Face-to-face on campus
- Structured or semi-structured interviews using prepared questions
- Notes taken manually or digitally

1. Objectives of the Interview

1. To understand **current campus processes** for lost & found items, notice dissemination, second-hand marketplace, college information, and feedback.
2. To identify **pain points and inefficiencies** in manual systems.
3. To assess the **interest and willingness** of users to adopt a digital campus management platform.
4. To gather **suggestions for features and improvements** to include in CampVerse.

2. Sample Interview Questions

A. Lost & Found System

1. Have you ever lost an item on campus? How did you retrieve it?
2. What challenges do you face in reporting or finding lost items?
3. Would a digital Lost & Found system help you? Why or why not?

B. Digital Notice Board

1. How do you currently receive important notices?
2. Do you miss any notices often?
3. Would you prefer a digital notice board with real-time updates?

C. Second-Hand Marketplace

1. Have you bought or sold used items on campus? How?
2. Do you think a campus-specific marketplace would make buying and selling easier and safer?

D. College Management System

1. How easy is it for you to access information about faculty, courses, and schedules?

2. What difficulties do you face while gathering college information?
3. Would an online College Management system help you?

E. Student Feedback System

1. How is feedback collected currently?
2. Do you feel comfortable giving feedback in the current system?
3. How can feedback collection be improved?

3. Method of Conducting the Interview

1. **Selection of Respondents:** A mix of 20–30 students, 5–10 faculty members, and 2–5 administrative staff.
2. **Setting:** Conducted in classrooms, faculty offices, and administrative offices.
3. **Duration:** Each interview lasted 10–15 minutes.
4. **Documentation:** Responses were recorded in notebooks or digital forms for analysis.
5. **Analysis:** Responses were analyzed to identify **common problems, feature requests, and usability preferences.**

4. Key Findings from Interviews

- **Lost & Found:** Students face delays and difficulty tracking lost items. A digital system is highly requested.
- **Notice Board:** Many students miss important notices; real-time digital updates are preferred.
- **Marketplace:** Students want a safe, organized, and campus-specific buying/selling platform.
- **College Information:** Difficulty in accessing faculty details and schedules; a centralized system is desired.
- **Feedback System:** Current feedback methods are inconvenient; an online anonymous feedback system is highly favored.

The interviews confirm that there is a **significant need for CampVerse**. Students and staff are eager for a **frontend-based digital platform** that simplifies campus management tasks, improves accessibility, and enhances communication. The findings from these interviews were used to **design, prioritize, and implement features** in the CampVerse project.

Requirement Analysis & Specification (Final Conclusion About the Information Gathering)

DATA FLOW DIAGRAM (DFD) - CampVerse

A **Data Flow Diagram (DFD)** is a graphical tool used to represent the flow of data within a system. It visually describes how information enters the system, how it is processed, where it moves, and how it is stored. For **CampVerse**, which integrates multiple campus services—Lost & Found System, Digital Notice Board, Second-hand Goods Marketplace, College Management System, and Student Feedback System—the DFD helps in understanding the overall workflow and interaction between different modules.

1. Purpose of DFD in CampVerse

The main purpose of a DFD in the CampVerse project is to:

- Represent the **movement of data** between students, faculty, and different system modules.
- Identify the **processes** involved in each subsystem.
- Show the **sources and destinations** of data.
- Help developers and students understand system flow clearly without technical backend details.
- Support the design of a structured, user-friendly, front-end-based campus solution.

2. Components Used in the CampVerse DFD

a) Process (Circle/Oval)

Represents the activities or operations performed by the system.

Examples in CampVerse:

- Lost & Found Management
- Goods Marketplace
- Feedback System
- College Management System
- Digital Notice Board

Each process transforms incoming data into useful output.

b) Data Flow (Arrows)

Shows the direction of data movement from source to destination.

Example:

- Student → submits feedback → Feedback System
- Faculty → provides notices → Digital Notice Board

c) External Entities (Rectangles)

Represent users or outside sources interacting with the system.
In CampVerse:

- Students
- Faculty
- College Administration

These entities initiate or receive data from system processes.

d) Data Stores (Open-ended Rectangles)

Used to store information temporarily or permanently.
Examples:

- Lost Item Records
- Notice Board Data
- Student Feedback Data
- Product Listings
- College Course Information

3. Importance of DFD in CampVerse

DFD plays an essential role in the development and documentation of CampVerse:

a) Clear Visualization

It helps visualize how different modules are connected.
For example, the Lost & Found Module interacts with Students and stores item data.

b) Better System Understanding

DFD provides a high-level and detailed-level understanding of the system for:

- Students
- Evaluators
- Faculty members

c) Simplifies System Design

Before implementing any interface, DFD ensures that the:

- Data flow is smooth
- Processes are well structured
- No important step is missed

d) Helps Identify Errors

By mapping the flow of data, missing or duplicate processes can be easily identified.

4. DFD Levels in CampVerse

a) Level 0 DFD (Context Diagram)

Represents the entire CampVerse system as a **single process**.

Shows only external users and data exchange.

Example Data Flow:

- Student → CampVerse → Notice, Feedback, Lost Item Updates
- Faculty → CampVerse → Feedback Records, Announcements

b) Level 1 DFD

Breaks down the main CampVerse system into its five major components:

1. Lost & Found System
2. Digital Notice Board
3. Second-Hand Goods Marketplace
4. College Management System
5. Student Feedback System

Shows how each subsystem interacts with users and data stores.

c) Level 2 DFD (Optional)

If required, each subsystem can be broken into smaller tasks.

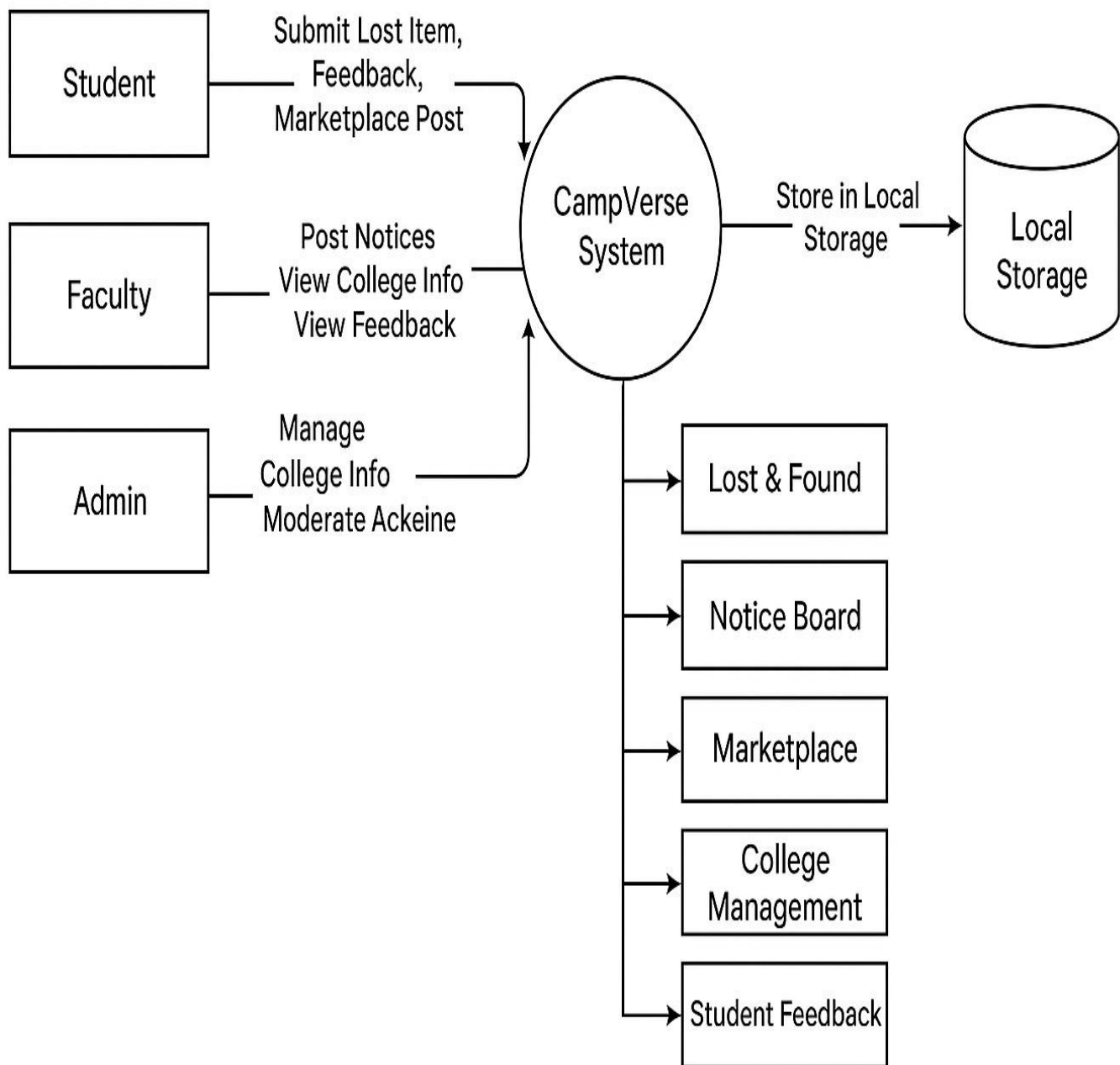
Example:

- Feedback System → Receive Feedback → Store Feedback → Display Summary

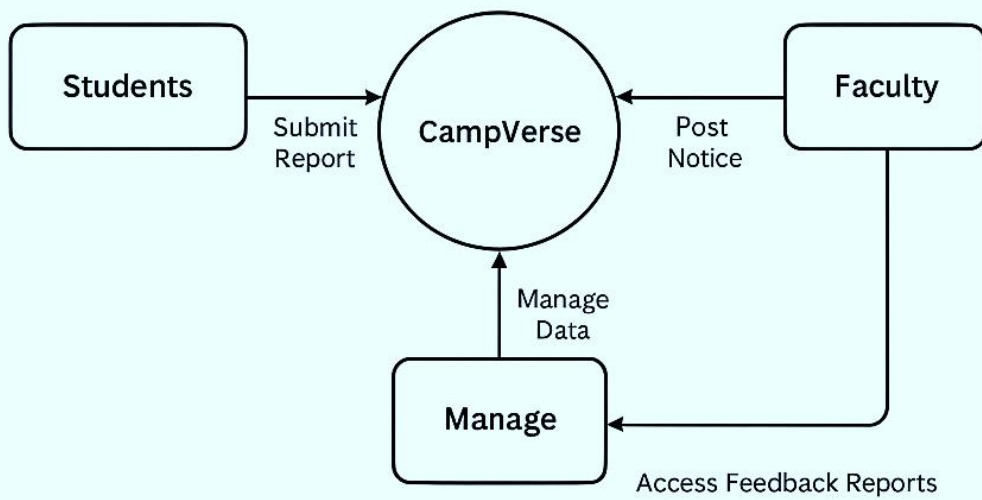
5. Benefits of Using DFD for CampVerse

- **Simple and easy** to understand for BCA project presentation.
- Clearly represents how information flows between different campus modules.
- Supports better documentation.

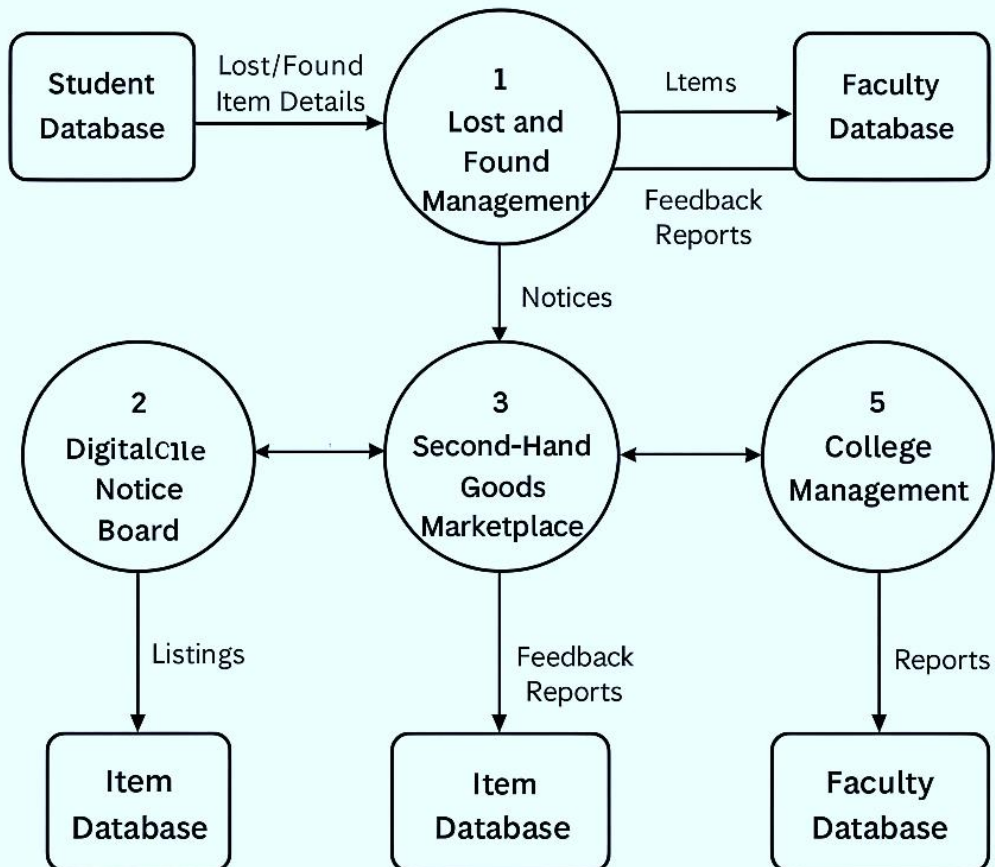
Data Flow Diagram (DFD) – CampVerse



Level 0



Level 1



Level 1 Data Flow Diagram

LOGICAL DESIGN

Entity Relationship Diagram (ERD) – CampVerse

The **Entity–Relationship Diagram (ERD)** is an important conceptual data modelling technique used to visually represent the structure of the CampVerse system. It shows the **entities**, their **attributes**, and the **relationships** between them. For CampVerse—which contains modules like Lost & Found Management, Digital Notice Board, Second-Hand Marketplace, College Management System, and Student Feedback System—the ERD helps in understanding how data will be organized and how different components interact with each other.

The purpose of the ER diagram in CampVerse is to create a **logical structure of the database**, even though the project does not include a backend. It still allows us to theoretically understand how data would flow and be stored if the system were implemented with a database. The ERD supports better system design, reduces redundancy, and highlights dependencies among entities.

Key Entities in CampVerse ERD

The ERD of CampVerse consists of the following major entities:

1. Student

The Student entity represents all users who use different modules of CampVerse.

Attributes: Student_ID (PK), Full_Name, Email, Phone.

A student can:

- Report lost items
- Post or buy goods in marketplace
- Submit feedback
- View digital notices
- View college-related information

2. Lost and Found Management

This entity stores details about items that students report as lost or found.

Attributes: Item_ID (PK), Item_Name, Description, Found_Date, Found_Location, Contact_Info, Status.

A single student can report multiple lost items → **1:N relationship**.

3. Digital Notice Board

This entity contains all notices published for the college.

Attributes: Notice_ID (PK), Title, Description, Date.

Notices are viewed by all students and faculty. Faculty/admin may post notices → **1:N relationship**.

4. Second-Hand Goods Marketplace

This entity manages items that students want to buy or sell.

Attributes: Product_ID (PK), Product_Name, Description, Price, Condition.

A student can post multiple products → **1:N relationship**.

5. College Management System

This entity stores basic information about college courses and departments.

Attributes: Course_ID (PK), Course_Name, Department.

Students belong to one course → **N:1 relationship** between Students and Courses.

6. Student Feedback System

This entity captures feedback submitted by students about faculty, services, or campus experiences.

Attributes: Feedback_ID (PK), Student_ID (FK), Faculty_ID (FK), Rating, Comments.

A student can give multiple feedbacks → **1:N relationship**.

7. Faculty

This entity stores faculty information for notices and feedback.

Attributes: Faculty_ID (PK), Name, Email, Department.

A faculty member can receive multiple feedback entries → **1:N relationship**.

Relationships in CampVerse ERD

✓ Student → Lost & Found (1:N)

A student can report multiple lost/found items.

✓ Faculty → Notice Board (1:N)

Faculty/Admin can post several notices.

✓ Student → Marketplace (1:N)

Students can list multiple items for sale.

✓ Student → Feedback (1:N)

A student may give feedback multiple times.

✓ Faculty → Feedback (1:N)

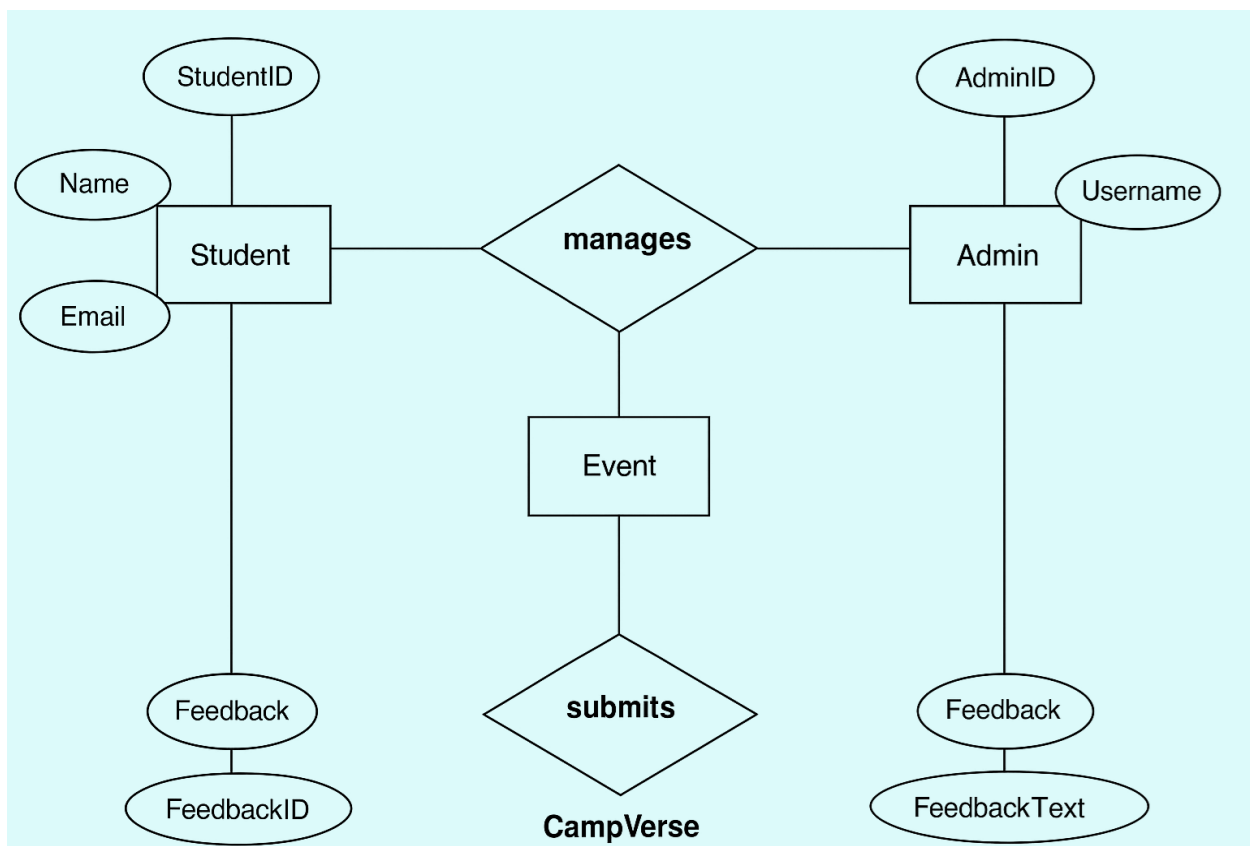
One faculty may receive many feedback entries.

✓ Student → College Course (N:1)

Many students are enrolled under one course.

Importance of ER Diagram in CampVerse

1. **Clarifies Data Structure:**
ERD provides a clear visual layout of all data used in CampVerse.
2. **Avoids Redundancy:**
Well-defined entities and relationships help reduce data duplication.
3. **Improves System Understanding:**
Helps developers, students, and teachers understand how various modules are linked.
4. **Acts as a Blueprint for Implementation:**
Even though CampVerse has no backend, ERD serves as a design model if the system is later expanded.
5. **Ensures Consistency and Accuracy:**
Shows proper entity linkage so the system works logically and efficiently.



ER DIAGRAM

DATA DICTIONARY – CampVerse

The **Data Dictionary** defines all data elements, attributes, and data structures used in the CampVerse system. It explains each field, its type, size, and purpose, ensuring consistency across all modules. Since CampVerse is frontend-based (no backend), the data dictionary reflects the information stored through **local storage** or temporary data structures handled by the user interface.

1. Student Entity

Field Name	Data Type	Size	Description
Student_ID	Integer	10	Unique identifier for each student
Full_Name	String	50	Name of the student
Email	String	50	Student email address
Phone	String	15	Contact number
Department	String	30	Department student belongs to

2. Faculty Entity

Field Name	Data Type	Size	Description
Faculty_ID	Integer	10	Unique faculty identifier
Faculty_Name	String	50	Name of the faculty member
Email	String	50	Faculty email address
Department	String	30	Assigned academic department

3. Lost & Found Management

Field Name	Data Type	Size	Description
Item_ID	Integer	10	Unique ID for each lost/found item
Item_Name	String	50	Name of the item
Description	String	200	Details about the item
Found_Location	String	100	Where the item was found
Found_Date	Date	—	Date item was found
Contact_Info	String	50	Contact of the person who found it
Status	String	15	Lost / Found / Returned

4. Digital Notice Board

Field Name	Data Type	Size	Description
Notice_ID	Integer	10	Unique notice identifier
Title	String	100	Title of the notice
Description	String	300	Content of the notice
Posted_By	String	50	Faculty/Admin posting the notice
Date	Date	—	Notice posting date

5. Second-Hand Goods Marketplace

Field Name	Data Type	Size	Description
Product_ID	Integer	10	Unique product identifier
Product_Name	String	50	Name of the item on sale
Description	String	200	Product description
Price	Float	10	Selling price
Condition	String	20	New/Good/Used
Seller_ID	Integer	10	Student who listed the item
Contact	String	30	Seller contact info

6. College Management System

a. Course Information

Field Name	Data Type	Size	Description
Course_ID	Integer	10	Unique course identifier
Course_Name	String	50	Name of the course
Department	String	30	Department offering the course

b. Attendance (If Included)

Field Name	Data Type	Size	Description
Attendance_ID	Integer	10	Unique attendance record ID
Student_ID	Integer	10	Student attending
Course_ID	Integer	10	Course attendance belongs to
Date	Date	—	Attendance date

7. Student Feedback System

Field Name	Data Type	Size	Description
Feedback_ID	Integer	10	Unique feedback identification number
Student_ID	Integer	10	Student giving feedback
Faculty_ID	Integer	10	Faculty receiving feedback
Rating	Integer	1	Rating between 1–5
Comment	String	200	Student's review or feedback text

8. Admin Entity

Field Name	Data Type	Size	Description
Admin_ID	Integer	10	Unique admin identifier
Name	String	50	Admin name
Email	String	50	Admin email for login/verification
Role	String	20	Admin / Moderator

DATABASE AND TABLE DESIGN - CampVerse

The CampVerse database is designed to store, organize, and manage all information related to its modules:

- Lost & Found Management
- Digital Notice Board
- Second-Hand Marketplace
- Student Feedback System
- College Management

The design ensures data integrity, normalization, and easy access between related tables through Primary Keys (PK) and Foreign Keys (FK).

Purpose:

Tracks transactions between buyers and sellers in the marketplace.

Used In: Second-Hand Marketplace module.

1. Relationships Summary

Relationship	Type	Description
Student ↔ LostAnd Found	1:N	A student can report many items.
Faculty ↔ Notice	1:N	A faculty member can post many notices.
Student ↔ Feedback	1:N	Each student can submit multiple feedbacks.
Faculty ↔ Feedback	1:N	Each faculty can receive multiple feedbacks.
Student ↔ Marketplace	1:N	A student can sell multiple products.
Student ↔ Transaction	N:M	Buyers and sellers are students.
Admin ↔ All Modules	1:N	Admin manages all data entities.

2. Design Highlights

- Normalization: All tables follow 3NF (Third Normal Form) to eliminate redundancy.
- Data Integrity: Maintained using primary keys and foreign key constraints.
- Scalability: Database supports future expansion like event modules or hostel management.
- Security: Passwords stored in encrypted form, and admin roles defined for access control.

PERT AND GANTT CHART – CampVerse

PERT CHART

The **Program Evaluation and Review Technique (PERT)** is a project management tool used to **plan, schedule, and control complex systems development**. It is especially useful for projects that involve multiple interconnected modules—such as **CampVerse**, which integrates five different systems:

1. Lost and Found Management System
2. Digital Notice Board
3. Second-Hand Goods Marketplace
4. College Management System
5. Student Feedback System

Because CampVerse contains several parallel tasks, dependencies, and multi-phase design work, a PERT chart helps ensure the project is completed efficiently and on time.

Purpose of Using PERT in CampVerse

Using a PERT chart for CampVerse helps in:

Handling multiple modules at once

Each module has separate tasks but also dependencies, such as UI designing before integration.

Estimating realistic timelines

PERT uses three-time estimates:

- **Optimistic (O)**
- **Most Likely (M)**
- **Pessimistic (P)**

This helps calculate a more accurate expected time for project completion.

Identifying critical paths

The **critical path** is the sequence of tasks that determines the **minimum project duration**.

For CampVerse, tasks like requirement analysis → UI design → module development → testing → integration usually fall on the critical path.

Managing uncertainty

If any module of CampVerse gets delayed (e.g., college management system UI), PERT shows how it impacts the entire project.

PERT Methodology Applied to CampVerse

Step-by-Step Approach

Step 1: Identify Major Tasks

Major project activities for CampVerse include:

1. Requirement Gathering
2. Preliminary Investigation
3. UI/UX Designing
4. Module Development (all 5 systems)
5. Frontend Integration
6. Testing & Validation
7. Documentation
8. Final Review & Submission

Step 2: Determine Task Dependencies

Examples:

- UI design must be completed **before development** begins.
- Development must be completed **before testing** begins.
- All modules must be tested before final integration.

This forms the foundation of the PERT flow.

Step 3: Construct the PERT Network

Tasks are arranged as nodes and arrows showing connections, such as:

Requirement Analysis → UI Design → Module Development → Testing → Integration → Documentation

Each module (Lost & Found, Notice Board, Marketplace, CMS, Feedback System) also has its own internal sequence but they run **parallel** to reduce overall duration.

Critical Path Analysis in CampVerse

The **critical path** is the longest sequence of dependent tasks that determine total project time.

For CampVerse, the likely critical path is:

Requirement Analysis → UI Design → Development of All Modules → Integration → System Testing → Documentation

If any task in this path gets delayed, the whole project deadline shifts.

Parallel tasks (e.g., marketplace and feedback system development) may have slack, meaning short delays won't affect the overall timeline—unless they block integration.

Benefits of PERT for CampVerse

✓ Better Time Management

It predicts accurate module completion time and helps in sequencing tasks.

✓ Efficient Resource Allocation

Team members can be assigned correctly based on task priority.

✓ Clear Visibility of Progress

The PERT structure shows where the project stands at any given time.

✓ Supports Parallel Workflows

Multiple CampVerse subsystems can be developed simultaneously without confusion.

✓ Minimizes Risk

By identifying critical tasks and slack time, the project team can mitigate delays.

PERT Application Summary for CampVerse

Project Stage	Why PERT Helps
Requirement Analysis	Helps define sequence of activities clearly
UI/UX Design	Shows dependency of all modules on design
Module Development	Parallel development scheduling
Integration	Identifies which module completion is critical
Testing	Ensures testing aligns with module delivery
Documentation	Ensures all tasks are completed before final write-up

GANTT CHART

A **Gantt Chart** is a project-management tool used to visually plan, schedule, and track the progress of tasks over a specific timeline. It displays activities on the vertical axis and the project timeline on the horizontal axis. Each task is represented by a horizontal bar, where the length of the bar indicates the duration of the activity.

For **CampVerse**, the Gantt chart helps organize development of all modules:

- Lost & Found Management System
- Digital Notice Board
- Second-Hand Goods Marketplace
- College Management System
- Student Feedback System

Since CampVerse is developed **without backend**, the focus is mainly on frontend UI design, planning, content structuring, interface workflow, and integration of multiple modules into one unified platform.

A Gantt chart ensures:

- proper distribution of work
- time efficiency
- clear understanding of responsibilities
- easy monitoring of project status
- well-structured project submission for academics

Text-Based Gantt Chart (October to December)

Task / Activity	Week 1 Oct	Week 2 Oct	Week 3 Oct	Week 4 Oct	Week 1 Nov	Week 2 Nov	Week 3 Nov	Week 4 Nov	Week 1 Dec	Week 2 Dec	Week 3 Dec
1. Project Planning & Requirement Analysis	✓	✓	✓								
2. Module Identification & Workflow Designing		✓	✓	✓							
3. UI Wireframing & Sketches			✓	✓	✓						
4. Lost & Found System UI Design				✓	✓	✓					
5. Digital Notice Board UI Design					✓	✓	✓				
6. Second-Hand Goods Marketplace UI						✓	✓	✓			
7. College Management System UI							✓	✓	✓		
8. Student Feedback System UI								✓	✓	✓	

PHYSICAL DESIGN

OUTPUT REQUIREMENT

Introduction

The **Physical Design** phase of the CampVerse system focuses on converting the logical design into a practical, user-interactive structure. Output requirements play a crucial role in ensuring that the information produced by the system is **clear, meaningful, accurate, and easy to understand** for all users, including students, faculty, administrators, and campus staff. In this phase, the emphasis is not only on *what* information the system will display but also *how* it will appear, how fast it will be delivered, and on which mediums it will be presented.

Purpose of Output Requirements

The main purpose of defining output requirements is to make sure that the system provides **useful and organized information** in the correct format. Since CampVerse consists of multiple modules such as Lost and Found Management, Digital Notice Board, Second-Hand Goods Marketplace, College Management, and Student Feedback System, each module needs to produce **timely and relevant outputs**. These outputs may include notifications, reports, summaries, listings, confirmations, alerts, and status updates. Well-designed outputs help users take quick decisions and improve campus operations.

Importance of Output Design

Output design is essential because it directly affects the **user experience**. If outputs are properly organized, visually clear, and easy to interpret, users can interact with the system smoothly. Additionally, effective output requirements reduce confusion, prevent errors, and increase system efficiency. For CampVerse, this includes outputs displayed on screens, digital boards, pop-ups, downloadable documents, and mobile-friendly views.

Scope of Output Requirements

The output requirements define the structure, layout, format, style, and delivery method of all information presented to users. It ensures that each module of CampVerse delivers the correct message at the right time in the right form.

USER INTERFACE DESIGN

User Interface (UI) Design focuses on crafting the visual layout and interactive elements through which users communicate with the CampVerse platform. Since CampVerse includes modules such as Lost and Found Management, Digital Notice Board, Second-Hand Goods Marketplace, College Management System, and Student Feedback System, the UI must remain intuitive, responsive, and consistent across all sections. A well-designed interface enhances usability, minimizes user effort, and ensures that students, faculty, and administrators can smoothly access the system's features without confusion.

1. Purpose of UI Design

The primary purpose of CampVerse's UI design is to create an environment where users can easily navigate different modules and perform required tasks efficiently. Whether students are browsing notices, reporting lost items, or submitting feedback, the UI should guide them logically. It must reduce cognitive load by using simple layouts, familiar icons, and clear labels.

2. Design Principles

To ensure effectiveness, CampVerse's UI follows key design principles such as **simplicity**, **consistency**, **visibility**, and **error prevention**. The interface avoids unnecessary elements and uses a minimalistic design approach to keep the system clutter-free. Consistent fonts, colors, and button styles help users develop familiarity across all modules.

3. User-Centered Approach

The UI is designed based on user requirements collected from students and faculty. Buttons, menus, search bars, and input forms are placed strategically to maximize accessibility. Special attention is given to mobile responsiveness since most students access the system via smartphones.

4. Visual Layout and Navigation

CampVerse uses a clean dashboard layout with module-specific icons that lead users directly to their tasks. Breadcrumb navigation, dropdown menus, and quick-access buttons enhance the browsing experience. Each module displays essential information prominently, such as notices, lost item reports, or feedback forms.

OUTPUT REPORT DESIGN

Output Report Design refers to the structured presentation of information generated by the CampVerse system. Since CampVerse consists of five major modules—**Lost & Found Management, Digital Notice Board, Second-Hand Goods Marketplace, College Management System, and Student Feedback System**—each module produces specific reports that help administrators, faculty, and students access meaningful and organized data. A well-designed output report ensures clarity, accuracy, readability, and quick decision-making.

1. Purpose of Output Report Design

The main purpose of designing output reports in CampVerse is to present processed information in a user-friendly format. These reports provide insights into student activities, item status, transaction history, feedback results, and campus announcements. The output reports allow stakeholders to evaluate system performance and track day-to-day operations efficiently.

2. Characteristics of Good Output Reports

CampVerse output reports are designed with the following characteristics:

✓ Clarity

Reports use simple layouts, clear headings, tables, and bullet points so information is easy to read.

✓ Accuracy

Only verified data entered by users is displayed to avoid confusion or errors.

✓ Relevance

Each report contains only essential information based on the selected module and user role.

✓ Consistency

Fonts, table styles, field names, and colors are uniform across all modules for a professional appearance.

✓ Timeliness

Reports are generated instantly, providing real-time insights and the latest updates.

3. Types of Output Reports in CampVerse

a) Lost & Found Report

- List of items found
- List of items claimed
- Item status (Unclaimed / Claimed)
- Submission & collection dates

b) Digital Notice Board Report

- All published notices
- Notice category report (Events, Exams, Alerts)
- Expired / upcoming notices

c) Second-Hand Goods Marketplace Report

- Product listing report
- Buyer–seller interaction summary
- Sold and available product report

d) College Management Report

- Student details report
- Faculty details report
- Course & department records
- Attendance summary (if included)

e) Student Feedback Report

- Individual feedback reports
- Course-wise feedback summary
- Faculty performance rating charts
- Overall satisfaction analysis

4. Output Format

CampVerse supports simple, readable formats, including:

- Tables
- Charts (for feedback reports)
- Item lists
- Status summaries
- Printable PDF-style layout

5. Conclusion

The Output Report Design of CampVerse ensures that every stakeholder receives clean, accurate, and well-structured information. By organizing reports for each module, CampVerse enhances decision-making, transparency, and campus management efficiency.

INPUT REQUIREMENT

Input requirements define all the data, fields, and information that the CampVerse system must receive from users to function effectively. Since CampVerse includes multiple modules—Lost & Found Management System, Digital Notice Board, Second-Hand Goods Marketplace, College Management System, and Student Feedback System—each module requires specific inputs to store, process, and generate accurate outputs. These inputs ensure smooth system operations, correct data handling, and efficient communication between users and the platform.

1. General User Inputs

These inputs are common for all students, faculty, and administrators:

- **User Name**
- **User ID / Enrollment Number**
- **Email Address**
- **Phone Number**
- **Department / Course**
- **User Role (Student / Faculty / Admin)**
- **Password (for authentication in the frontend environment)**
(Note: Even without backend, password input is required for UI design.)

2. Lost & Found Management System Inputs

This module requires accurate item-related data:

a) Lost Item Reporting Inputs

- Item Name
- Item Description
- Lost Date & Time
- Lost Location
- Student Name & Contact
- Uploaded Image (optional)

b) Found Item Reporting Inputs

- Item Name
- Item Description
- Found Date & Time
- Found Location
- Finder Name & Contact
- Image of the item

3. Digital Notice Board Inputs

Admins must enter the following to post notices:

- Notice Title
- Notice Category (Exam, Event, Circular, Urgent, etc.)
- Notice Description
- Uploaded PDF/Image (optional)
- Post Date
- Valid Till / Expiry Date

4. Second-Hand Goods Marketplace Inputs

To list products for sale or exchange, students provide:

- Product Name
- Product Category (Books, Gadgets, Accessories, etc.)
- Product Description
- Price
- Condition (New / Used / Slightly Used)
- Seller Name & Contact
- Product Image

5. College Management System Inputs

For institutional data handling, the following inputs are required:

a) Student Information Inputs

- Name
- Course
- Year/Semester
- Roll Number
- Attendance Records
- Academic Details

b) Faculty Information Inputs

- Name
- Department
- Designation
- Contact Details

c) Administrative Inputs

- Class Schedules
- Examination Timetable
- Event Information

6. Student Feedback System Inputs

To maintain structured feedback collection, students must input:

- Student Name / ID
- Faculty Name
- Course Name
- Rating (1–5 scale)
- Comments / Suggestions

7. System-Level Inputs

These inputs help with interface navigation and feature selection:

- Menu Selection
- Button Clicks
- Search Queries
- Filter Options
- File Upload Inputs

Conclusion

The input requirements of CampVerse ensure that each module receives the correct and complete information needed to generate meaningful outputs. Well-defined inputs reduce errors, support an intuitive user experience, and help maintain system accuracy—even without backend integration.

PROJECT IMPLEMENTATION (CODING)

campVerse_Backend	4/12/2026 12:09 AM	File folder	
CampVerse_Frontend	4/4/2026 11:13 PM	File folder	
.gitignore	4/4/2026 11:12 PM	Git Ignore Source File	2 KB

Frontend Structure:-

node_modules	4/4/2026 11:15 PM	File folder	
public	4/4/2026 11:12 PM	File folder	
src	4/4/2026 11:12 PM	File folder	
.gitignore	4/4/2026 11:12 PM	Git Ignore Source File	1 KB
eslint.config	4/4/2026 11:12 PM	JavaScript Source File	1 KB
index	4/4/2026 11:12 PM	Microsoft Edge HTM...	1 KB
package	4/4/2026 11:12 PM	JSON Source File	2 KB
package-lock	4/4/2026 11:14 PM	JSON Source File	141 KB
postcss.config	4/4/2026 11:12 PM	JavaScript Source File	1 KB
README	4/4/2026 11:12 PM	Markdown Source File	2 KB
tailwind.config	4/4/2026 11:12 PM	JavaScript Source File	1 KB
vite.config	4/4/2026 11:12 PM	JavaScript Source File	1 KB
vite	4/4/2026 11:12 PM	Microsoft Edge HTM...	2 KB

Backend Structure:-

campverse	4/4/2026 11:27 PM	File folder	
college	4/4/2026 11:27 PM	File folder	
feedback	4/4/2026 11:27 PM	File folder	
lostfound	4/4/2026 11:27 PM	File folder	
marketplace	4/4/2026 11:27 PM	File folder	
media	4/4/2026 11:37 PM	File folder	
notices	4/4/2026 11:27 PM	File folder	
users	4/4/2026 11:27 PM	File folder	
venv	4/4/2026 11:21 PM	File folder	
db.sqlite3	4/12/2026 12:09 AM	SQLITE3 File	
manage	4/4/2026 11:12 PM	Python Source File	
requirements	4/4/2026 11:12 PM	Text Document	

Process Design

Coding

Frontend :-

```
//src/main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "../App.jsx";
import "../index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);

<!-- index.html -->
<!doctype html>
<html lang="en" >
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>campverse</title>
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link
      href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&display=swap"
      rel="stylesheet"
    />

    </head>
    <body>
      <div id="root"></div>
      <script type="module" src="/src/main.jsx"></script>
    </body>

  </html>

// This component checks if the user is logged in by looking for a token in localStor
import { Navigate } from "react-router-dom";

const ProtectedRoute = ({ children }) => {
  const isLoggedIn = !!localStorage.getItem("campverse_token");
  if (!isLoggedIn) {
    return <Navigate to="/login" replace />;
  }
  return children;
};

export default ProtectedRoute;
```

```

//src/App.jsx
import { Routes, Route } from "react-router-dom";
import ProtectedRoute from "../components/ProtectedRoute";
import Home from "../pages/Home";
import Login from "../pages/auth/LoginPage";
import Register from "../pages/auth/Register";
// import ForgotPassword from "../pages/auth/ForgotPassword";
import Dashboard from "../pages/Dashboard";
import CollegeManagement from "../pages/CollegeManagement";
import LostFound from "../pages/LostFound";
import Notices from "../pages/Notices";
import Marketplace from "../pages/Marketplace";
import Feedback from "../pages/Feedback";

function App() {
  return (
    <Routes>
      <Route path="/login" element={<Login />} />
      { /* <Route path="/register" element={<Register />} /> */ }
      { /* <Route path="/forgot-password" element={<ForgotPassword />} /> */ }
      <Route path="/" element={
        <ProtectedRoute>
          <Home />
        </ProtectedRoute>
      } />
      <Route path="/dashboard" element={
        <ProtectedRoute>
          <Dashboard />
        </ProtectedRoute>
      } />
      <Route path="/college-management" element={
        <ProtectedRoute>
          <CollegeManagement />
        </ProtectedRoute>
      } />
      <Route path="/lost-found" element={
        <ProtectedRoute>
          <LostFound />
        </ProtectedRoute>
      } />
      <Route path="/notices" element={
        <ProtectedRoute>
          <Notices />
        </ProtectedRoute>
      } />
      <Route path="/marketplace" element={
        <ProtectedRoute>
          <Marketplace />
        </ProtectedRoute>
      } />
      <Route path="/feedback" element={
        <ProtectedRoute>
          <Feedback />
        </ProtectedRoute>
      } />
    </Routes>
  );
}

export default App;

```

```
// package.json
```

```
{
  "name": "campverse",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "@popperjs/core": "^2.11.8",
    "@tanstack/react-query": "^5.90.19",
    "axios": "^1.13.4",
    "bootstrap": "^5.3.8",
    "framer-motion": "^12.29.0",
    "lucide-react": "^0.562.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^7.13.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.1",
    "@types/react": "^19.2.5",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^5.1.1",
    "autoprefixer": "^10.4.23",
    "eslint": "^9.39.1",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.4.24",
    "globals": "^16.5.0",
    "postcss": "^8.5.6",
    "tailwindcss": "^3.4.19",
    "vite": "npm:rolldown-vite@7.2.5"
  },
  "overrides": {
    "vite": "npm:rolldown-vite@7.2.5"
  }
}
```

```

/* //src/index.css */
@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  @apply bg-slate-50 text-slate-900;
  font-family: "Poppins", system-ui, -apple-system, BlinkMacSystemFont,
    "Segoe UI", sans-serif;
}
@keyframes authFadeIn {
  0% {
    opacity: 0;
    transform: translateY(16px) scale(0.98);
  }
  100% {
    opacity: 1;
    transform: translateY(0) scale(1);
  }
}
@keyframes noticePop {
  0% {
    opacity: 0;
    transform: translateY(6px) scale(0.98);
  }
  100% {
    opacity: 1;
    transform: translateY(0) scale(1);
  }
}

@keyframes noticeFormIn {
  0% {
    opacity: 0;
    transform: translateY(24px);
  }
  100% {
    transform: translate(0, 0) scale(1);
    opacity: 0.7;
  }
}

@keyframes bubbleRight {
  0% {
    transform: translate(0, 0) scale(1);
    opacity: 0.7;
  }
  50% {
    transform: translate(-24px, -18px) scale(1.06);
    opacity: 1;
  }
  100% {
    transform: translate(0, 0) scale(1);
    opacity: 0.7;
  }
}
@keyframes noticePop {
  0% { opacity: 0; transform: translateY(6px) scale(0.98); }
  100% { opacity: 1; transform: translateY(0) scale(1); }
}

```

```

/* //src/App.css */
#root {
  max-width: 1280px;
  margin: 0 auto;
  padding: 2rem;
  text-align: center;
}

.logo {
  height: 6em;
  padding: 1.5em;
  will-change: filter;
  transition: filter 300ms;
}
.logo:hover {
  filter: drop-shadow(0 0 2em #646cffff);
}
.logo.react:hover {
  filter: drop-shadow(0 0 2em #61dafbaa);
}

@keyframes logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}

@media (prefers-reduced-motion: no-preference) {
  a:nth-of-type(2) .logo {
    animation: logo-spin infinite 20s linear;
  }
}

// src/components/Navbar.jsx - Only Search + Bell Active
import { useState, useEffect } from "react";
import { NavLink, Link, useNavigate } from "react-router-dom";
import api from "../api/client";

const Navbar = () => {
  // const [showSearch, setShowSearch] = useState(false);
  const [showNotifications, setShowNotifications] = useState(false);
  // const [searchQuery, setSearchQuery] = useState("");
  const [notifications, setNotifications] = useState([]);
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();

  const isLoggedIn = !!localStorage.getItem("campverse_token");

  // Fetch notifications on mount
  useEffect(() => {
    fetchNotifications();
  }, []);

```

```
const fetchNotifications = async () => {
  try {
    setLoading(true);
    const response = await api.get("/api/notices/");
    if (response.data && Array.isArray(response.data)) {
      // Get latest 5 notices as notifications
      const latestNotices = response.data.slice(0, 5).map(notice => ({
        id: notice.id,
        title: notice.title,
        description: notice.description?.substring(0, 50) + "..." || "",
        category: notice.category,
        timeAgo: notice.time_ago,
        isPinned: notice.is_pinned
      }));
      setNotifications(latestNotices);
    }
  }
}
```

```

} catch (error) {
  console.error("Error fetching notifications:", error);
  // Fallback to demo data if API fails
  setNotifications([
    { id: 1, title: "Welcome to Campverse", description: "Explore all features...", category: "General", timeAgo: "Just now" },
  ]);
} finally {
  setLoading(false);
}
};

const handleLogout = async () => {
  try {
    await api.post("/api/auth/logout/");
  } catch (err) {
    console.error(err);
  }
  localStorage.removeItem("campverse_token");
  localStorage.removeItem("campverse_user");
  navigate("/login");
};

const base = "text-sm font-medium text-slate-600 hover:text-slate-900 transition-colors";
const active = "text-slate-900";

// const handleSearch = async (e) => {};

return (
  <nav className="bg-white/90 backdrop-blur border-b border-slate-100 sticky top-0 z-30 dark:bg-slate-900/90 dark:border-slate-600">
    <div className="max-w-6xl mx-auto px-4 py-3 flex items-center justify-between gap-4">
      { /* Logo */ }
      <NavLink to="/" className="flex items-center gap-2">
        <span className="text-xl font-extrabold bg-gradient-to-r from-sky-500 via-indigo-500 to-orange-400 bg-clip-text text-transparent">

```

```

    className="w-full mt-3 py-2 text-xs text-center text-blue-600 hover:text-blue-700 dark:text-blue-400 dark:ho
  >
    View all notifications
  </button>
  })
</div>
})
</div>

{/* Search Icon - REMOVED */}

{/* Login / Logout */}
{isLoggedIn ? (
  <button
    onClick={handleLogout}
    className="px-4 py-2 rounded-full bg-red-500 text-white text-sm font-semibold shadow-sm hover:bg-red-600 hover:scale-1
  >
    Logout
  </button>
) : (
  <Link
    to="/login"
    className="px-4 py-2 rounded-full bg-blue-600 text-white text-sm font-semibold shadow-sm hover:bg-blue-700 hover:scale-1
  >
    Login
  </Link>
)}
</div>
</div>
</nav>
);
};
export default Navbar;

```

```

//src/components/Footer.jsx
import { NavLink } from "react-router-dom";

const linkClass =
  "text-xs text-slate-300 hover:text-white transition-colors duration-200";
const activeClass = "text-white font-semibold";

const Footer = () => {
  return (
    <footer className="mt-12 bg-slate-950 text-slate-200">
      <div className="max-w-6xl mx-auto px-4 py-10 md:py-12 space-y-8">
        <div className="grid gap-8 md:grid-cols-3">
          {/* Brand */}
          <div className="space-y-3">
            <h3 className="text-lg font-semibold text-white">Campverse</h3>
            <p className="text-xs text-slate-400 leading-relaxed">
              Your comprehensive campus platform connecting students,
              services and community. Making campus life easier, one click
              at a time.
            </p>
            <div className="flex items-center gap-3 text-lg mt-3">
              <button className="h-8 w-8 rounded-full bg-slate-900 flex items-center justify-center hover:bg-slate-800 transition-c
                <span></span>
              </button>
              <button className="h-8 w-8 rounded-full bg-slate-900 flex items-center justify-center hover:bg-slate-800 transition-c
                <span></span>
              </button>
              <button className="h-8 w-8 rounded-full bg-slate-900 flex items-center justify-center hover:bg-slate-800 transition-c
                <span>in</span>
              </button>
            </div>
          </div>
        </div>

        {/* Quick links */}
        <div className="space-y-3">
          <h4 className="text-sm font-semibold text-white">Quick Links</h4>

```



```

const CampusGallery = () => {
  const [active, setActive] = useState(1);

  // auto-change after 5s
  useEffect(() => {
    const timer = setInterval(() => {
      setActive((prev) => (prev === slides.length ? 1 : prev + 1));
    }, 5000);
    return () => clearInterval(timer);
  }, []);

  const goPrev = () => {
    setActive((prev) => (prev === 1 ? slides.length : prev - 1));
  };

  const goNext = () => {
    setActive((prev) => (prev === slides.length ? 1 : prev + 1));
  };

  <div className="grid grid-cols-3 md:grid-cols-4 gap-4 pt-4">
    {[
      { label: "Campus Events", value: "500+" },
      { label: "Happy Students", value: "1000+" },
      { label: "Campus Facilities", value: "50+" },
      { label: "Campus Access", value: "24/7" },
    ].map((stat, idx) => (
      <div
        key={idx}
        className="
          text-center py-4 rounded-2xl
          bg-white/10 backdrop-blur-xl
          border border-purple-100/70
          shadow-sm shadow-purple-200/70
          transition-all duration-300 ease-out
          hover:-translate-y-1 hover:shadow-lg hover:shadow-purple-300
        "
      >
        <p className="text-2xl md:text-3xl font-bold text-purple-700">
          {stat.value}
        </p>
        <p className="text-xs text-slate-700 mt-3">
          {stat.label}
        </p>
      </div>
    ))}
  </div>
</div>
</section>
);
};

export default CampusGallery;

```

```
//src/pages/CollegeManagement.jsx
import MainLayout from "../layouts/MainLayout";
import React, { useState, useEffect } from "react";
import api from "../components/api/client";
import { motion, AnimatePresence } from "framer-motion";
import {
  Calendar,
  Users,
  DollarsSign,
  BookOpen,
  Clock,
  BarChart3,
} from "lucide-react";

const CollegeManagement = () => {
  const [activeTab, setActiveTab] = useState("dashboard");
  const [loading, setLoading] = useState(true);

  // ----- Data from API -----
  const [courses, setCourses] = useState([]);
  const [faculty, setFaculty] = useState([]);
  const [timetable, setTimetable] = useState([]);
  const [attendance, setAttendance] = useState([]);
  const [fees, setFees] = useState([]);

  // ----- Dashboard stats -----
  const totalStudents = courses.reduce((sum, c) => sum + (c.students || 0), 0);
  const totalFeesCollected = fees.filter(f => f.status === 'Paid').reduce((sum, f) => sum + parseFloat(f.amount), 0);
  const totalFeesPending = fees.filter(f => f.status === 'Pending').reduce((sum, f) => sum + parseFloat(f.amount), 0);

  const todayAttendance = attendance.length > 0 ? attendance[attendance.length - 1] : null;
  const attendancePercent = todayAttendance && todayAttendance.total > 0
    ? Math.round((todayAttendance.present / todayAttendance.total) * 100)
    : 0;

  const handleCourseSubmit = async (e) => {
    e.preventDefault();
    if (!courseForm.name.trim()) return;
    try {
      if (editingCourse) {
        await api.put(`/api/college/courses/${editingCourse.id}/`, courseForm);
      } else {
        await api.post('/api/college/courses/', courseForm);
      }
      fetchData();
      setShowCourseForm(false);
      setEditingCourse(null);
    } catch (err) {
      console.error("Error saving course:", err);
      alert("Error saving course");
    }
  };

  const handleCourseDelete = async (id) => {
    if (!window.confirm("Delete this course?")) return;
    try {
      await api.delete(`/api/college/courses/${id}/`);
      fetchData();
    } catch (err) {
      console.error("Error deleting course:", err);
    }
  };
};

```

```

    { /* Fee Collection */ }
    <div className="rounded-3xl p-6 border border-orange-100 bg-gradient-to-br from-orange-50 via-white to-purple-50">
      <h3 className="text-sm font-semibold text-slate-800 mb-4">
        | 🕒 Fee Collection Status
      </h3>
      <div className="space-y-4">
        <div>
          <div className="flex justify-between text-xs mb-1">
            <span className="text-slate-600">Collected</span>
            <span className="font-semibold text-emerald-600">₹18.5L (82%)</span>
          </div>
          <div className="h-3 bg-slate-200 rounded-full overflow-hidden">
            <div className="h-full w-[82%] bg-gradient-to-r from-emerald-400 to-emerald-500 rounded-full" />
          </div>
        </div>
        <div>
          <div className="flex justify-between text-xs mb-1">
            <span className="text-slate-600">Pending</span>
            <span className="font-semibold text-orange-600">₹4.2L (18%)</span>
          </div>
          <div className="h-3 bg-slate-200 rounded-full overflow-hidden">
            <div className="h-full w-[18%] bg-gradient-to-r from-orange-400 to-orange-500 rounded-full" />
          </div>
        </div>
      </div>
      <div>
        <div className="mt-6 grid grid-cols-2 gap-4">
          <div className="bg-white rounded-xl p-3 shadow-sm">
            <p className="text-xs text-slate-500">Total Revenue</p>
            <p className="text-lg font-bold text-slate-800">₹22.7L</p>
          </div>
          <div className="bg-white rounded-xl p-3 shadow-sm">
            <p className="text-xs text-slate-500">Pending Dues</p>
            <p className="text-lg font-bold text-orange-600">₹4.2L</p>
          </div>
        </div>
      </div>
    </div>

    { /* Recent Activity */ }
    <div className="mt-6 rounded-3xl p-6 border border-slate-100 bg-white">
      <h3 className="text-sm font-semibold text-slate-800 mb-4">

<div className="flex items-center justify-end gap-2 pt-2">
  <button
    type="button"
    onClick={() => {
      setShowFeeForm(false);
      setEditingFee(null);
    }}
    className="px-4 py-2 rounded-xl border border-slate-200 bg-white text-xs font-medium text-slate-600 hover:bg-slate-50"
  >
    Cancel
  </button>
  <button
    type="submit"
    className="px-4 py-2 rounded-xl bg-gradient-to-r from-indigo-500 to-purple-600 text-xs font-semibold text-white shadow-[
  >
    <div className="flex justify-between text-xs mb-1">
      <span className="font-semibold text-emerald-600">₹18.5L (82%)</span>
    </div>
    <div className="h-3 bg-slate-200 rounded-full overflow-hidden">
      <div className="h-full w-[82%] bg-gradient-to-r from-emerald-400 to-emerald-500 rounded-full" />
    </div>
  </div>
  <div>
    <div className="flex justify-between text-xs mb-1">
      <span className="font-semibold text-orange-600">₹4.2L (18%)</span>
    </div>
    <div className="h-3 bg-slate-200 rounded-full overflow-hidden">
      <div className="h-full w-[18%] bg-gradient-to-r from-orange-400 to-orange-500 rounded-full" />
    </div>
  </div>
</div>
</div>

    { /* Recent Activity */ }
    <div className="mt-6 rounded-3xl p-6 border border-slate-100 bg-white">
      <h3 className="text-sm font-semibold text-slate-800 mb-4">

```

```

        {editingFee ? "" : ""}
      </button>
    </div>
  </form>
</motion.div>
</motion.div>
  )}
</AnimatePresence>
</motion.div>
)}
</AnimatePresence>
</motion.main>
</div>
</div>
</MainLayout>
);
};

```

```
export default CollegeManagement;
```

```
//src/pages/Dashboard.jsx
```

```
import MainLayout from "../layouts/MainLayout";
```

```

const Dashboard = () => {
  return (
    <MainLayout>
      <div className="space-y-6">
        <header className="flex items-center justify-between gap-3">
          <div>
            <h1 className="text-2xl font-semibold">Dashboard</h1>
            <p className="text-slate-600 text-sm">
              Quick overview of attendance, classes and upcoming events.
            </p>
          </div>
          <button className="px-3 py-2 rounded-lg bg-blue-600 text-white text-xs font-medium hover:bg-blue-700">
            Download Report
          </button>
        </header>

        <section className="grid md:grid-cols-4 gap-4">
          {[
            { label: "Present Today", value: "842", color: "bg-blue-50" },
            { label: "Absent Today", value: "37", color: "bg-rose-50" },
            { label: "Active Courses", value: "64", color: "bg-emerald-50" },
            { label: "New Notices", value: "5", color: "bg-amber-50" },
          ].map((card, idx) => (
            <div
              key={idx}
              className={` ${card.color} rounded-xl p-4 border border-slate-100` }
            >
              <p className="text-xs text-slate-500">{card.label}</p>
              <p className="text-2xl font-semibold mt-1">{card.value}</p>
            </div>
          ))}
        </section>

        <section className="grid md:grid-cols-2 gap-6">
          <div className="bg-white rounded-xl shadow-sm p-4 border border-slate-100">
            <h2 className="text-sm font-semibold mb-3">Today's Classes</h2>
            <ul className="space-y-2 text-sm">
              <li className="flex items-center justify-between">
                <span>DBMS <span>CSE 3rd Year</span>
                <span className="px-2 py-1 rounded-full bg-emerald-50 text-emerald-700 text-xs">
                  09:00 AM
                </span>
              </li>
            </ul>
          </div>
        </section>
      </MainLayout>
    )
  );
};

```

```

    <li className="flex items-center justify-between">
      <span>Physics & BSc 1st Year</span>
      <span className="px-2 py-1 rounded-full bg-violet-50 text-violet-700 text-xs">
        02:00 PM
      </span>
    </li>
  </ul>
</div>

<div className="bg-white rounded-xl shadow-sm p-4 border border-slate-100">
  <h2 className="text-sm font-semibold mb-3">Upcoming Events</h2>
  <ul className="space-y-2 text-sm">
    <li className="flex items-center justify-between">
      <span>Tech Fest 2025</span>
      <span className="text-xs text-slate-500">28 Dec</span>
    </li>
    <li className="flex items-center justify-between">
      <span>Internal Exams</span>
      <span className="text-xs text-slate-500">02 Jan</span>
    </li>
    <li className="flex items-center justify-between">
      <span>Placement Drive</span>
      <span className="text-xs text-slate-500">15 Jan</span>
    </li>
  </ul>
</div>
</section>
</div>
</MainLayout>
  );
};

export default Dashboard;

```

```

//src/pages/Home.jsx
import { useEffect } from "react";
import { useNavigate } from "react-router-dom";
import MainLayout from "../layouts/MainLayout";
import CampusGallery from "../components/CampusGallery";

const Home = () => {
  const navigate = useNavigate();

  return (
    <MainLayout>
      <div className="min-h-screen bg-slate-50 text-slate-900 dark:bg-slate-950 dark:text-slate-100 transition-colors duration-300">
        <div className="py-10 md:py-14 relative overflow-hidden">
          <div className="hero-bubbles pointer-events-none absolute inset-0 -z-10" />

          <div className="max-w-5xl mx-auto text-center space-y-6 px-4">
            <div className="inline-flex items-center px-4 py-1.5 rounded-full bg-blue-50 text-xs font-medium text-blue-700 border border-blue-100">
              <span className="mr-2">🌟</span> Your Campus, Connected
            </div>

            <h1 className="text-4xl md:text-5xl font-extrabold tracking-tight text-slate-900 dark:text-slate-50 space-y-1">
              <span className="block">Welcome to</span>
              <span className="block bg-gradient-to-r from-sky-400 via-indigo-500 to-orange-400 bg-clip-text text-transparent">
                Campverse
              </span>
            </h1>

            <p className="max-w-3xl mx-auto text-slate-600 dark:text-slate-300 text-sm md:text-base">
              All in one campus platform for lost items, notices, marketplace, feedback and complaint management. Connect, share and thrive together.
            </p>

            <div className="flex items-center justify-center gap-3 mt-2">
              <button
                onClick={() => navigate("/lost-found")}
                className="px-6 py-3 rounded-full bg-gradient-to-r from-blue-500 via-indigo-500 to-sky-400 text-white text-sm font-semibold shadow-lg shadow-blue-400/40 transition-all duration-300 ease-out">
                Lost & Found
              </button>

              <div className="absolute left-3 top-3 h-8 w-8 rounded-2xl bg-white/90 dark:bg-slate-900/90 shadow flex items-center">
                🏠
              </div>
            </div>

            <div className="p-4 space-y-2 flex-1 flex flex-col">
              <h3 className="text-base font-semibold text-slate-900 dark:text-slate-50">College Management</h3>
              <p className="text-xs text-slate-600 dark:text-slate-300">Dashboard for attendance, timetable, fees and student records.</p>
              <ul className="mt-1 space-y-1 text-xs text-slate-600 dark:text-slate-300">
                <li>✓ Student profiles & course details</li>
                <li>✓ Attendance & timetable overview</li>
                <li>✓ Role-based access (admin, faculty, students)</li>
              </ul>
              <button
                onClick={() => navigate("/college-management")}
                className="mt-3 w-full py-2 rounded-xl bg-blue-600 text-white text-xs font-semibold hover:bg-blue-700 transition-colors">
                View Management Module
              </button>
            </div>
          </div>
        </div>
      </MainLayout>
    </div>
  );
};

export default Home;

```

```

7/src/pages/LostFound.jsx
import { useEffect, useState } from "react";
import MainLayout from "../../layouts/MainLayout";
import api from "../../components/api/client";

const LostFound = () => {
  const [items, setItems] = useState([]);
  const [query, setQuery] = useState("");
  const [statusFilter, setStatusFilter] = useState("All");
  const [mounted, setMounted] = useState(false);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchItems();
    const t = setTimeout(() => setMounted(true), 80);
    return () => clearTimeout(t);
  }, []);

  const fetchItems = async () => {
    try {
      const res = await api.get('/api/lostfound/');
      setItems(res.data);
    } catch (err) {
      console.error("Error fetching items:", err);
    } finally {
      setLoading(false);
    }
  };

  const filtered = items.filter((it) => {
    const matchText =
      it.title.toLowerCase().includes(query.toLowerCase()) ||
      (it.description || "").toLowerCase().includes(query.toLowerCase()) ||
      (it.location || "").toLowerCase().includes(query.toLowerCase());

    const matchStatus =
      statusFilter === "All" ? true : it.status === statusFilter;

    return matchText && matchStatus;
  });

  const handleSubmit = async (e) => {
    e.preventDefault();
    const formData = new FormData(e.currentTarget);
    const newItem = {
      title: formData.get("title"),
      description: formData.get("description"),
      location: formData.get("location"),
      contact_info: formData.get("email"),
      status: "Lost",
      date: new Date().toISOString().split('T')[0],
    };
    if (!newItem.title) return;

```



```

<div className="space-y-4 max-h-[420px] overflow-y-auto pr-1">
  {filtered.map((it) => (
    <article key={it.id} className="bg-white rounded-2xl border border-slate-100 shadow-[0_12px_30px_rgba(15,23,42,0.04)] px-4 py-4 md:px-5 md:py-4 flex flex-col md:flex-row md:items-center jus
      <div className="space-y-1">
        <h3 className="text-sm md:text-base font-semibold text-slate-900">{it.title}</h3>
        <p className="text-xs md:text-sm text-slate-600">{it.description}</p>
        <div className="flex flex-wrap items-center gap-4 text-xs text-slate-500 mt-2">
          <span className="flex items-center gap-1">📍 {it.location}</span>
          <span className="flex items-center gap-1">📅 {it.date}</span>
        </div>
      </div>
      <div className="flex items-end md:items-center gap-3 md:flex-col">
        <span className="inline-flex items-center justify-center px-3 py-1 rounded-full text-[11px] font-semibold ${it.status === "Lost" ? "bg-gradient-to-r from-rose-500/10 to-rose-500/25 tex
          {it.status}
        </span>
        <div className="flex gap-2">
          <button onClick={() => handleContact(it)} className="px-4 py-2 rounded-xl border border-slate-200 text-xs md:text-sm font-medium text-slate-700 bg-slate-50/80 hover:bg-slate-100 trans
            Contact
          </button>
          <button onClick={() => handleDelete(it.id)} className="px-4 py-2 rounded-xl border border-rose-200 text-xs md:text-sm font-medium text-rose-600 bg-rose-50/70 hover:bg-rose-100 transit
            Delete
          </button>
        </div>
      </div>
    </article>
  )})
  {filtered.length === 0 && (
    <p className="text-xs text-slate-500 text-center py-6">No items match your search yet.</p>
  )}
</div>
</section>
</div>
</div>
</MainLayout>
);
};

```

```
export default LostFound;
```

```
//src/pages/Marketplace.jsx
```

```

import { useEffect, useMemo, useState } from "react";
import MainLayout from "../layouts/MainLayout";
import api from "../components/api/client";

```

```
const categories = ["All", "Electronics", "Books", "Furniture", "Sports", "Clothing"];
```

```

const initialItems = [
  {
    id: 1,
    title: 'MacBook Pro 13" M1',
    category: "Electronics",
    condition: "Like New",
    price: 85000,
    oldPrice: 129900,
    seller: "Arjun Sharma",
    location: "Room 305, Block A",
    posted: "2 hours ago",
    rating: 4.8,
    tags: ["Laptop", "Apple", "Programming"],
    image: "https://images.unsplash.com/photo-1517336714731-489689fd1ca8?w=600&auto=format&fit=crop&q=60&ixlib=rb-4.1.0&i
  },

```

```

const Marketplace = () => {
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [query, setQuery] = useState("");
  const [activeCategory, setActiveCategory] = useState("All");
  const [showForm, setShowForm] = useState(false);
  const [highlightId, setHighlightId] = useState(null);

```

```

// Sell form fields
const [formTitle, setFormTitle] = useState("");
const [formCategory, setFormCategory] = useState("Electronics");
const [formCondition, setFormCondition] = useState("Good");
const [formPrice, setFormPrice] = useState("");
const [formSeller, setFormSeller] = useState("");
const [formLocation, setFormLocation] = useState("");
const [formTags, setFormTags] = useState("");
const [formImage, setFormImage] = useState(null);
const [imagePreview, setImagePreview] = useState(null);

<label className="text-xs font-medium text-slate-600">
  Tags (comma separated)
</label>
<input
  type="text"
  value={formTags}
  onChange={(e) => setFormTags(e.target.value)}
  placeholder="e.g. Laptop, Apple, 256GB"
  className="
    w-full px-3 py-2 rounded-xl
    border border-slate-200 bg-slate-50
    focus:outline-none focus:ring-2 focus:ring-emerald-500/80 focus:bg-white
  "
/>
</div>

<div className="flex items-center justify-end gap-2 pt-2">
  <button
    type="button"
    onClick={() => setShowForm(false)}
    className="
      px-4 py-2 rounded-xl
      border border-slate-200 bg-white text-xs font-medium text-slate-600
      hover:bg-slate-50
    "
  >
    Cancel
  </button>
  <button
    type="submit"
    className="
      px-4 py-2 rounded-xl
      bg-emerald-500 text-xs font-semibold text-white
      shadow-sm shadow-emerald-300/70
      hover:bg-emerald-600 hover:shadow-md hover:shadow-emerald-400/80
      transition-all duration-200
    "
  >
    Submit
  </button>
</div>

```

```
    >  
      Post Item  
    </button>  
  </div>  
</form>  
</div>  
</div>  
  })  
</div>  
})  
</MainLayout>  
);  
};  
  
export default Marketplace;
```

```
// src/pages/Notices.jsx
import { useEffect, useState } from "react";
import MainLayout from "../layouts/MainLayout";
import api from "../components/api/client";

const tagColors = {
  Academics: "bg-blue-50 text-blue-600",
  General: "bg-slate-100 text-slate-700",
  Hostel: "bg-purple-50 text-purple-600",
  Events: "bg-pink-50 text-pink-600",
};

const Notices = () => {
  const [notices, setNotices] = useState([]);
  const [loading, setLoading] = useState(true);

  const [showForm, setShowForm] = useState(false);
  const [pendingNotice, setPendingNotice] = useState(null);
  const [mounted, setMounted] = useState(false);

  // form fields
  const [title, setTitle] = useState("");
  const [body, setBody] = useState("");
  const [tag, setTag] = useState("General");

  // filters
  const [search, setSearch] = useState("");
  const [activeTag, setActiveTag] = useState("All");

  // page entry animation
  useEffect(() => {
    const t = setTimeout(() => setMounted(true), 60);
    return () => clearTimeout(t);
  }, []);

  // backend se notices fetch karo
  useEffect(() => {
    const fetchNotices = async () => {
      try {
        // Django notices app me tum list endpoint banaoge, maine example "/api/notices/active/" rakha tha.[file:2]
        const res = await api.get("/api/notices/active/");
        // Assume backend response array of objects: { id, title, body/text, category/tag, created_at/timeAgo ... }
        const apiNotices = res.data.map((n) => ({
          id: n.id,
          title: n.title,
          body: n.description || n.body || "",
          tag: n.category || n.tag || "General",
          timeAgo: n.time_ago || "Just now",
        }));
        setNotices(apiNotices);
      } catch (err) {
        console.error("Error fetching notices", err);
      } finally {

```

```

{/* Slide-in notice form (drawer style) */}
{showForm && (
  <div className="fixed inset-0 z-40 flex items-end md:items-center justify-center bg-slate-900/40 backdrop-blur-sm">
    <div
      className="
        w-full md:max-w-lg
        bg-white rounded-t-3xl md:rounded-3xl
        shadow-2xl border border-slate-100
        animate-[noticeFormIn_0.35s_ease-out]
      "
    >
      <div className="flex items-center justify-between px-5 pt-4 pb-2 border-b border-slate-100">
        <div>
          <h2 className="text-sm font-semibold text-slate-900">
            | Post New Notice
          </h2>
          <p className="text-xs text-slate-500">
            | Share important updates with all students.
          </p>
        </div>
        <button
          onClick={() => setShowForm(false)}
          className="h-8 w-8 rounded-full bg-slate-100 text-slate-500 text-xs flex items-center justify-center hover:bg-slate-200"
        >
          X
        </button>
      </div>

      <form
        onSubmit={handleSubmit}
        className="px-5 py-4 space-y-3 text-sm"
      >
        <div className="space-y-1">
          <label className="text-xs font-medium text-slate-600">
            | Title
          </label>
          <input
            type="text"
            value={title}
            onChange={(e) => setTitle(e.target.value)}
            placeholder="e.g. Internal Assessment schedule updated"
            className="
              w-full px-3 py-2 rounded-xl
              border border-slate-200 bg-slate-50
              focus:outline-none focus:ring-2 focus:ring-indigo-500/80 focus:bg-white
            "
          />
        </div>
      </form>
    </div>
  </div>
)
}

```

```

        type="submit"
        className="
            px-4 py-2 rounded-xl
            bg-gradient-to-r from-indigo-500 via-sky-500 to-emerald-400
            text-xs font-semibold text-white
            shadow-sm shadow-indigo-300/70
            hover:shadow-md hover:shadow-indigo-400/80
            transition-all duration-200
        "
    >
        Post Notice
    </button>
</div>
</form>
</div>
</div>
    )}
</div>
    )}
</MainLayout>
    );
};

```

```
export default Notices;
```

```

// src/api/client.js
import axios from "axios";

const API_BASE_URL = "http://127.0.0.1:8000"; // Django ka URL

const api = axios.create({
    baseURL: API_BASE_URL,
    withCredentials: true, // Send cookies for CSRF
});

// request interceptor: har request pe token laga do
api.interceptors.request.use((config) => {
    const token = localStorage.getItem("campverse_token");
    if (token) {
        config.headers.Authorization = `Token ${token}`;
    }
    return config;
});

export default api;

```

```
//src/layouts/MainLayout.jsx
```

```
import { Link } from "react-router-dom";
import Navbar from "../components/Navbar";
import Footer from "../components/Footer";

const MainLayout = ({ children }) => {
  return (
    <div className="min-h-screen flex flex-col bg-slate-50 dark:bg-slate-950 text-slate-900 dark:text-slate-100 transition-all duration-300">
      { /* Navbar */ }
      <Navbar />

      { /* Main Content - Full width with proper padding */ }
      <main className="flex-1 w-full">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8 lg:py-12">
          {children}
        </div>
      </main>

      { /* Footer */ }
      <Footer />
    </div>
  );
};
```

```
//src/pages/auth/AuthLayout.jsx
```

```
import { Link } from "react-router-dom";

const AuthLayout = ({ title, subtitle, children }) => {
  return (
    <div className="min-h-screen bg-slate-50 text-slate-900 flex dark:bg-slate-950 dark:text-slate-50 transition-colors duration-300">
      { /* Left gradient panel */ }
      <div className="hidden lg:flex w-1/2 relative overflow-hidden">
        <div className="absolute inset-0 bg-gradient-to-br from-indigo-500 via-sky-500 to-emerald-400" />
        <div className="absolute inset-0 bg-[radial-gradient(circle_at_top_left,rgba(255,255,255,0.25),transparent_60%)]" />
        <div className="relative z-10 flex flex-col justify-between p-10">
          <Link to="/" className="inline-flex items-center gap-2">
            <div className="h-9 w-9 rounded-2xl bg-white/10 border border-white/30 flex items-center justify-center text-xl">
              🏠
            </div>
            Home
          </Link>
        </div>
      </div>

      <div className="space-y-6">
        <p className="text-xs font-semibold uppercase tracking-[0.2em] text-sky-100">
          Smart Campus Portal
        </p>
        <h2 className="text-3xl font-bold leading-snug">
          One account for all your{" "}
          <span className="text-sky-100">campus essentials.</span>
        </h2>
        <ul className="space-y-2 text-sm text-sky-50/90">
          <li>• Track attendance, notices & lost items</li>
          <li>• Buy & sell second-hand books and supplies</li>
          <li>• Share feedback and raise complaints in seconds</li>
        </ul>
      </div>
    </div>
  );
};
```

```

    { /* Heading */}
    <div className="mb-6 text-center">
      <h1 className="text-2xl sm:text-3xl font-bold text-slate-900">
        {title}
      </h1>
      {subtitle && (
        <p className="mt-1 text-xs sm:text-sm text-slate-500">
          {subtitle}
        </p>
      )}
    </div>

    { /* Card (white) */}
    <div
      className="
        bg-white dark:bg-slate-900
        border border-slate-200 dark:border-slate-700
        rounded-2xl px-5 py-6 sm:px-6 sm:py-7
        shadow-[0_18px_45px_rgba(15,23,42,0.12)] dark:shadow-[0_18px_45px_rgba(0,0,0,0.35)]
        backdrop-blur-xl
      "
    >
      {children}
    </div>
  </div>
</div>
</div>
);
};

```

```
export default AuthLayout;
```

```
//src/pages/auth/LoginPage.jsx
```

```
import { useEffect, useState } from "react"; ...
```

```

const LoginPage = ({ onLogin }) => {
  const navigate = useNavigate();
  const [view, setView] = useState("login");
  const [loading, setLoading] = useState(false);
  const [showPassword, setShowPassword] = useState(false);
  const [formData, setFormData] = useState({
    name: "",
    email: "",
    password: "",
    confirmPassword: "",
  });

  useEffect(() => {
    const link = document.createElement("link");
    link.href = "https://fonts.googleapis.com/css2?family=Poppins:wght@400;500;600;700&display=swap";
    link.rel = "stylesheet";
    document.head.appendChild(link);
  }, []);

  const handleChange = (event) => {
    setFormData((previous) => ({ ...previous, [event.target.name]: event.target.value }));
  };

  const handleLogin = async (event) => {
    event.preventDefault();
    if (!formData.email || !formData.password) {
      alert("Please enter campus email and password");
      return;
    }
  }
}

```



```

<form onSubmit={handleLogin} className="space-y-5">
  <div>
    <label className="block text-sm font-semibold text-slate-700 dark:text-slate-200 mb-2">Campus Email</label>
    <input name="email" value={formData.email} onChange={handleChange} type="email" className={fieldClass} placeholder="you@college.edu" required />
  </div>
  <div>
    <label className="block text-sm font-semibold text-slate-700 dark:text-slate-200 mb-2">Password</label>
    <div className="relative">
      <input name="password" value={formData.password} onChange={handleChange} type={showPassword ? "text" : "password"} className={` ${fieldClass} pr-12`} placeholder={showPassword ? "Show Password" : "Hide Password"} />
      <button type="button" onClick={() => setShowPassword(current => !current)} className="absolute right-4 top-4 text-slate-400 dark:text-slate-500 hover:text-slate-500">{showPassword ? "Hide" : "Show"}</button>
    </div>
  </div>
  <button type="submit" disabled={loading} className={`w-full py-4 rounded-xl text-white font-bold transition-all shadow-lg hover:shadow-xl ${loading ? "bg-slate-500" : "bg-slate-700"}>{loading ? "Signing in..." : "Sign in"}</button>
  <div className="mt-6 space-y-4 text-center">
    <button type="button" onClick={() => setView("forgot")} className="text-sm text-sky-500 hover:text-sky-600 font-medium dark:text-sky-400 dark:hover:text-sky-500">Forgot password?</button>
    <div className="border-t border-slate-200 dark:border-slate-700 pt-4">
      <button type="button" onClick={() => setView("signup")} className="text-sm text-slate-600 hover:text-slate-900 font-medium dark:text-slate-300 dark:hover:text-slate-400">Don't have an account?</button>
      <span className="text-sky-500 font-semibold dark:text-sky-400">Create one</span>
    </div>
  </div>
</form>
)}

<form onSubmit={handleForgotPassword} className="space-y-5">
  <div>
    <label className="block text-sm font-semibold text-slate-700 dark:text-slate-200 mb-2">Campus Email</label>
    <input name="email" value={formData.email} onChange={handleChange} type="email" className={fieldClass} placeholder="you@college.edu" required />
  </div>
  <button type="submit" className="w-full py-4 rounded-xl bg-gradient-to-r from-sky-600 to-indigo-500 text-white font-bold hover:from-sky-700 hover:to-indigo-600">Send reset link</button>
  <div className="text-center">
    <button type="button" onClick={() => setView("login")} className="text-sm text-slate-600 hover:text-slate-900 font-medium dark:text-slate-300 dark:hover:text-slate-400">Back to login</button>
  </div>
</form>
)}

<p className="mt-6 text-[11px] text-slate-400 text-center">
  © {new Date().getFullYear()} Campverse
</p>
</AuthLayout>
)};
};

export default LoginPage;

//src/pages/auth/ForgotPassword.jsx
import { Link } from "react-router-dom";
import AuthLayout from "../AuthLayout";

const ForgotPassword = () => {
  const handleSubmit = (e) => {
    e.preventDefault();
    // yahan password reset email API
  };

  return (
    <AuthLayout
      title="Reset your password"
      subtitle="Enter your college email and we'll send you reset instructions."
    >

```

```

    <button
      type="submit"
      className="
        w-full py-2.5 rounded-xl
        bg-gradient-to-r from-purple-500 via-indigo-500 to-sky-500
        text-sm font-semibold text-white
        shadow-[0_14px_45px_rgba(129,140,248,0.6)]
        transition-all duration-300
        hover:shadow-[0_18px_55px_rgba(59,130,246,0.8)]
        hover:translate-y-0.5
      "
    >
      Send reset link
    </button>

    <p className="text-[11px] text-slate-400 text-center">
      Remember your password?{" " }
      <Link
        to="/login"
        className="text-sky-400 hover:text-sky-300 font-medium"
      >
        Back to login
      </Link>
    </p>
  </form>
</AuthLayout>
);
};

export default ForgotPassword;

```

```

// src/pages/auth/Register.jsx
import { Link } from "react-router-dom";
import AuthLayout from "../AuthLayout";

const Register = () => {
  const handleSubmit = (e) => {
    e.preventDefault();
    // yahan signup API call
  };

  return (
    <AuthLayout
      title="Create your campus account"
      subtitle="Use your official college email to sign up."
    >
      <form onSubmit={handleSubmit} className="space-y-4">
        <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
          <div className="space-y-1">
            <label className="text-xs font-medium text-slate-300">
              Full Name
            </label>
            <input
              type="text"
              required
              placeholder="e.g. Rohan Sharma"
              className="
                w-full px-3 py-2.5 text-sm rounded-xl
                bg-slate-900 border border-slate-700
                text-slate-100 placeholder:text-slate-500
                focus:outline-none focus:ring-2 focus:ring-sky-500 focus:border-sky-500
              "
            />
          </div>
        </div>
      </form>
    </AuthLayout>
  );
};

export default Register;

```

```

<div className="space-y-1">
  <label className="text-xs font-medium text-slate-300">
    Roll / ID
  </label>
  <input
    type="text"
    required
    placeholder="e.g. CSE-21-045"
    className="
      w-full px-3 py-2.5 text-sm rounded-xl
      bg-slate-900 border border-slate-700
      text-slate-100 placeholder:text-slate-500
      focus:outline-none focus:ring-2 focus:ring-sky-500 focus:border-sky-500
    "
  />
</div>
</div>

```

```

<div className="space-y-1">
  <label className="text-xs font-medium text-slate-300">
    College Email
  </label>
  <input
    type="email"
    required
    placeholder="you@college.edu"
    className="
      w-full px-3 py-2.5 text-sm rounded-xl
      bg-slate-900 border border-slate-700
      text-slate-100 placeholder:text-slate-500
      focus:outline-none focus:ring-2 focus:ring-sky-500 focus:border-sky-500
    "
  />
</div>

```

```

<div className="flex items-start gap-2 text-[11px] text-slate-400">
  <input
    type="checkbox"
    required
    className="mt-0.5 h-3.5 w-3.5 rounded border-slate-600 bg-slate-900"
  />
  <span>
    I agree to the campus usage policy and data guidelines.
  </span>
</div>

<button
  type="submit"
  className="
    w-full py-2.5 rounded-xl
    bg-gradient-to-r from-emerald-400 via-sky-500 to-indigo-500
    text-sm font-semibold text-slate-950
    shadow-[0_14px_45px_rgba(45,212,191,0.45)]
    transition-all duration-300
    hover:shadow-[0_18px_55px_rgba(56,189,248,0.65)]
    hover:translate-y-0.5
  "
>
  Create account
</button>

<p className="text-[11px] text-slate-400 text-center">
  Already have an account?{" "}
  <Link
    to="/login"
    className="text-sky-400 hover:text-sky-300 font-medium"
  >
    Log in
  </Link>
</p>
</form>
</AuthLayout>
);
};

```

```
export default Register;
```

Backend:-

```
# Django settings for the CampVerse project, including configurations for installed apps, middleware, database, authentication,
from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/4.2/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-quv8o&1v59m)3!mqd=k3o@y*be=%@%#pd*c0u17ny!jdh_@wih'

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = ['*']

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
]

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.TokenAuthentication',
        'rest_framework.authentication.SessionAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.AllowAny',
    ],
}

ROOT_URLCONF = 'campverse.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]

WSGI_APPLICATION = 'campverse.wsgi.application'

# Database
# https://docs.djangoproject.com/en/4.2/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

```

# URL configuration for the CampVerse project, routing API endpoints for authentication, notices, marketplace, lost and found, college, and feedback applica
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/auth/', include('users.urls')),
    path('api/notices/', include('notices.urls')),
    path('api/marketplace/', include('marketplace.urls')),
    path('api/lostfound/', include('lostfound.urls')),
    path('api/college/', include('college.urls')),
    path('api/feedback/', include('feedback.urls')),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)

# Admin configuration for the college application, registering models such as Course, Faculty, Timetable, Attendance, and Fee
from django.contrib import admin...

@admin.register(Course)
class CourseAdmin(admin.ModelAdmin):
    list_display = ['name', 'credits', 'students', 'fees', 'created_at']
    search_fields = ['name']

@admin.register(Faculty)
class FacultyAdmin(admin.ModelAdmin):
    list_display = ['name', 'subject', 'department', 'email', 'phone']
    search_fields = ['name', 'subject', 'department']

@admin.register(Timetable)
class TimetableAdmin(admin.ModelAdmin):
    list_display = ['day', 'course', 'slot', 'room', 'faculty']
    list_filter = ['day', 'course']

@admin.register(Attendance)
class AttendanceAdmin(admin.ModelAdmin):
    list_display = ['date', 'course', 'present', 'total', 'status']
    list_filter = ['date', 'course', 'status']

@admin.register(Fee)
class FeeAdmin(admin.ModelAdmin):
    list_display = ['student_name', 'course', 'amount', 'status', 'date']
    list_filter = ['status', 'course', 'date']
    search_fields = ['student_name']

# apps.py for the college application, defining the CollegeConf
from django.apps import AppConfig
class CollegeConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'college'

```

```
# Models for the college application, including Course, Faculty, Timetable, Attendance, and Fee models to manage various aspects of the college system.
```

```
from django.db import models
from django.conf import settings

class Course(models.Model):
    name = models.CharField(max_length=255)
    credits = models.IntegerField(default=4)
    students = models.IntegerField(default=0)
    fees = models.DecimalField(max_digits=10, decimal_places=2, default=0)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return self.name

class Faculty(models.Model):
    name = models.CharField(max_length=255)
    subject = models.CharField(max_length=255)
    phone = models.CharField(max_length=20, blank=True)
    email = models.EmailField(blank=True)
    department = models.CharField(max_length=100, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return self.name

class Timetable(models.Model):
    DAYS = [
        ('Monday', 'Monday'),
        ('Tuesday', 'Tuesday'),
        ('Wednesday', 'Wednesday'),
        ('Thursday', 'Thursday'),
        ('Friday', 'Friday'),
        ('Saturday', 'Saturday'),
    ]
    day = models.CharField(max_length=20, choices=DAYS)
    course = models.ForeignKey(Course, on_delete=models.CASCADE, related_name='timetables')
    slot = models.CharField(max_length=50)
    room = models.CharField(max_length=50)
    faculty = models.ForeignKey(Faculty, on_delete=models.CASCADE, related_name='timetables', null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f"{self.day} - {self.course.name}"
```

```
from rest_framework import serializers
from .models import Course, Faculty, Timetable, Attendance, Fee
```

```
class CourseSerializer(serializers.ModelSerializer):
    class Meta:
        model = Course
        fields = ['id', 'name', 'credits', 'students', 'fees', 'created_at', 'updated_at']
        read_only_fields = ['id', 'created_at', 'updated_at']
```

```
class FacultySerializer(serializers.ModelSerializer):
    class Meta:
        model = Faculty
        fields = ['id', 'name', 'subject', 'phone', 'email', 'department', 'created_at', 'updated_at']
        read_only_fields = ['id', 'created_at', 'updated_at']
```

```
class TimetableSerializer(serializers.ModelSerializer):
    course_name = serializers.CharField(source='course.name', read_only=True)
    faculty_name = serializers.CharField(source='faculty.name', read_only=True)

    class Meta:
        model = Timetable
        fields = ['id', 'day', 'course', 'course_name', 'slot', 'room', 'faculty', 'faculty_name', 'created_at', 'updated_at']
        read_only_fields = ['id', 'created_at', 'updated_at']

    def validate_course(self, value):
        # Handle course from form (could be int, string, or null)
        if value is None or value == '' or value == 'null':
            raise serializers.ValidationError("Course is required")
        if isinstance(value, str):
            try:
                return int(value)
            except ValueError:
                raise serializers.ValidationError("Invalid course ID")
        return value
```

```
# URL configuration for the college application, defining
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from . import views
```

```
router = DefaultRouter()
router.register('courses', views.CourseViewSet)
router.register('faculty', views.FacultyViewSet)
router.register('timetable', views.TimetableViewSet)
router.register('attendance', views.AttendanceViewSet)
router.register('fees', views.FeeViewSet)
```

```
urlpatterns = [
    path('', include(router.urls)),
]
```

```
# Admin configuration for the lost and found application, registering the LostFoundItem model with the
```

```
from django.contrib import admin
from .models import LostFoundItem
@admin.register(LostFoundItem)
class LostFoundItemAdmin(admin.ModelAdmin):
    list_display = ['title', 'status', 'location', 'date', 'posted_by', 'is_resolved', 'created_at']
    list_filter = ['status', 'is_resolved', 'created_at']
    search_fields = ['title', 'description', 'location']
    list_editable = ['is_resolved']
    ordering = ['-created_at']
```

```
# Models for the lost and found application, defining the LostFoundItem model which includes fields for title, descriptio
```

```
from django.db import models
from django.conf import settings

class LostFoundItem(models.Model):
    STATUS_CHOICES = [
        ('Lost', 'Lost'),
        ('Found', 'Found'),
    ]

    title = models.CharField(max_length=255)
    description = models.TextField(blank=True)
    status = models.CharField(max_length=10, choices=STATUS_CHOICES)
    location = models.CharField(max_length=255)
    date = models.DateField()
    contact_info = models.CharField(max_length=255, blank=True)
    image = models.ImageField(upload_to='lostfound_images/', blank=True, null=True)
    posted_by = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, related_name='lostfound_items')
    is_resolved = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-created_at']

    def __str__(self):
        return f"{self.title} ({self.status})"
```

```
# Serializers for the lost and found application, converting LostFoundItem model instances to JSON format for API responses and validating input data for creating and up
from rest_framework import serializers
from .models import LostFoundItem
```

```
class LostFoundItemSerializer(serializers.ModelSerializer):
    posted_by_name = serializers.CharField(source='posted_by.get_full_name', read_only=True)

    class Meta:
        model = LostFoundItem
        fields = ['id', 'title', 'description', 'status', 'location', 'date', 'contact_info', 'image', 'posted_by', 'posted_by_name', 'is_resolved', 'created_at', 'updated_at']
        read_only_fields = ['id', 'posted_by', 'is_resolved', 'created_at', 'updated_at']
```

```

# A D:\MajorProject\campverse\campVerse_Backend\marketplace - Contains emphasized items the Item model with the Django admin interface to allow for easy
from django.contrib import admin
from .models import Item

@admin.register(Item)
class ItemAdmin(admin.ModelAdmin):
    list_display = ['title', 'category', 'condition', 'price', 'seller', 'status', 'created_at']
    list_filter = ['category', 'condition', 'status', 'created_at']
    search_fields = ['title', 'description', 'seller_username']
    list_editable = ['status']
    ordering = ['-created_at']

# URL configuration for the lost and found application, defining API endpoints for managing lost and found items using
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from . import views

router = DefaultRouter()
router.register('', views.LostFoundItemViewSet)

urlpatterns = [
    path('', include(router.urls)),
]

# Views for the lost and found application, defining the LostFoundItemViewSet which provides API endpoints for managing lost and found items.
from rest_framework import viewsets, permissions
from .models import LostFoundItem
from .serializers import LostFoundItemSerializer

class LostFoundItemViewSet(viewsets.ModelViewSet):
    queryset = LostFoundItem.objects.all()
    serializer_class = LostFoundItemSerializer
    permission_classes = [permissions.AllowAny]

    def perform_create(self, serializer):
        serializer.save(posted_by=self.request.user)

    def get_queryset(self):
        queryset = super().get_queryset()
        status = self.request.query_params.get('status')

        if status and status != 'All':
            queryset = queryset.filter(status=status)

        return queryset

```



```

# Models for the marketplace application, defining the Item model which includes fields for title, description, category, condition, price
from django.db import models
from django.conf import settings

class Item(models.Model):
    CATEGORY_CHOICES = [
        ('Electronics', 'Electronics'),
        ('Books', 'Books'),
        ('Furniture', 'Furniture'),
        ('Sports', 'Sports'),
        ('Clothing', 'Clothing'),
    ]

    CONDITION_CHOICES = [
        ('Like New', 'Like New'),
        ('Good', 'Good'),
        ('Fair', 'Fair'),
        ('Poor', 'Poor'),
    ]

    STATUS_CHOICES = [
        ('Available', 'Available'),
        ('Sold', 'Sold'),
        ('Pending', 'Pending'),
    ]

    title = models.CharField(max_length=255)
    description = models.TextField(blank=True)
    category = models.CharField(max_length=20, choices=CATEGORY_CHOICES)
    condition = models.CharField(max_length=20, choices=CONDITION_CHOICES, default='Good')
    price = models.DecimalField(max_digits=10, decimal_places=2)
    old_price = models.DecimalField(max_digits=10, decimal_places=2, blank=True, null=True)
    seller = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.SET_NULL, null=True, blank=True, related_name='items')
    location = models.CharField(max_length=255, blank=True)
    image = models.ImageField(upload_to='marketplace_images/', blank=True, null=True)
    tags = models.JSONField(default=list, blank=True)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='Available')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

# Serializers for the marketplace application, converting Item model instances to JSON format for API responses and validating input data for creating and updating marketplace items
from rest_framework import serializers
from django.utils import timezone
from .models import Item

class ItemSerializer(serializers.ModelSerializer):
    seller_name = serializers.CharField(source='seller.get_full_name', read_only=True)
    seller_email = serializers.EmailField(source='seller.email', read_only=True)
    posted = serializers.SerializerMethodField()
    image = serializers.ImageField(required=False, allow_null=True)

    class Meta:
        model = Item
        fields = ['id', 'title', 'description', 'category', 'condition', 'price', 'old_price', 'seller', 'seller_name', 'seller_email', 'location', 'image', 'tags', 'status',
            read_only_fields = ['id', 'seller', 'status', 'created_at', 'updated_at']

    def get_posted(self, obj):
        delta = timezone.now() - obj.created_at
        if delta.seconds < 3600:
            return f'{delta.seconds // 60} min ago'
        elif delta.days == 0:
            return f'{delta.seconds // 3600} hr ago'
        elif delta.days == 1:
            return '1 day ago'
        else:
            return f'{delta.days} days ago'

class ItemCreateSerializer(serializers.ModelSerializer):
    image = serializers.ImageField(required=False, allow_null=True)
    old_price = serializers.CharField(required=False, allow_null=True, allow_blank=True)
    tags = serializers.CharField(required=False, allow_blank=True)

    class Meta:
        model = Item
        fields = ['title', 'description', 'category', 'condition', 'price', 'old_price', 'location', 'image', 'tags']

    def to_internal_value(self, data):
        # Convert form data to proper types before validation
        ret = super().to_internal_value(data)

# URL configuration for the marketplace application, defining API endpoints for managing marketplace items using Django REST
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from . import views

router = DefaultRouter()
router.register('', views.ItemViewSet)

urlpatterns = [
    path('', include(router.urls)),
]

```

```

# Views for the marketplace application, defining the ItemViewSet which provides API endpoints for managing marketplace items. The viewset allows for creating,
from rest_framework import viewsets, permissions
from rest_framework.response import Response
from rest_framework.exceptions import ValidationError
from .models import Item
from .serializers import ItemSerializer, ItemCreateSerializer
import logging

logger = logging.getLogger(__name__)

class ItemViewSet(viewsets.ModelViewSet):
    queryset = Item.objects.all()
    serializer_class = ItemSerializer
    permission_classes = [permissions.AllowAny]

    def get_serializer_class(self):
        if self.action == 'create':
            return ItemCreateSerializer
        return ItemSerializer

    def create(self, request, *args, **kwargs):
        serializer = self.get_serializer(data=request.data)
        try:
            serializer.is_valid(raise_exception=True)
            self.perform_create(serializer)
            headers = self.get_success_headers(serializer.data)
            logger.info(f"Successfully created item: {serializer.data}")
            return Response(serializer.data, status=201, headers=headers)
        except ValidationError as e:
            logger.error(f"Validation error creating item: {e.detail}")
            logger.error(f"Request data was: {request.data}")
            raise

    def perform_create(self, serializer):
        # Save with seller if authenticated, otherwise without
        if self.request.user.is_authenticated:
            serializer.save(seller=self.request.user)
        else:
            serializer.save()

    def get_queryset(self):
        queryset = super().get_queryset()
        category = self.request.query_params.get('category')

# Admin configuration for the notices application, registering the Notice model with the Django admin interface to allow for easy management of notices. The
from django.contrib import admin
from .models import Notice

@admin.register(Notice)
class NoticeAdmin(admin.ModelAdmin):
    list_display = ['title', 'category', 'author', 'is_pinned', 'is_active', 'created_at']
    list_filter = ['category', 'is_pinned', 'is_active', 'created_at']
    search_fields = ['title', 'description']
    list_editable = ['is_pinned', 'is_active']
    ordering = ['-is_pinned', '-created_at']

# Models for the notices application, defining the Notice model which includes fields for title, description, category, author (linked to the
from django.db import models
from django.conf import settings

class Notice(models.Model):
    CATEGORY_CHOICES = [
        ('Academics', 'Academics'),
        ('General', 'General'),
        ('Hostel', 'Hostel'),
        ('Events', 'Events'),
    ]

    title = models.CharField(max_length=255)
    description = models.TextField(blank=True)
    category = models.CharField(max_length=20, choices=CATEGORY_CHOICES, default='General')
    author = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.SET_NULL, null=True, related_name='notices')
    is_pinned = models.BooleanField(default=False)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-is_pinned', '-created_at']

    def __str__(self):
        return self.title

```

```
# Serializers for the notices application, converting Notice model instances to JSON format for API responses and validating input data for creating and up
> from rest_framework import serializers...
```

```
class NoticeSerializer(serializers.ModelSerializer):
    author_name = serializers.CharField(source='author.get_full_name', read_only=True)
    time_ago = serializers.SerializerMethodField()

    class Meta:
        model = Notice
        fields = ['id', 'title', 'description', 'category', 'author', 'author_name', 'is_pinned', 'is_active', 'created_at', 'updated_at', 'time_ago']
        read_only_fields = ['id', 'author', 'created_at', 'updated_at']

    def get_time_ago(self, obj):
        delta = timezone.now() - obj.created_at
        if delta.seconds < 3600:
            return f"{delta.seconds // 60} min ago"
        elif delta.days == 0:
            return f"{delta.seconds // 3600} hr ago"
        else:
            return f"{delta.days} day(s) ago"

class NoticeCreateSerializer(serializers.ModelSerializer):
    class Meta:
        model = Notice
        fields = ['title', 'description', 'category', 'is_pinned', 'is_active']
```

```
# Models for the marketplace application, defining the Item model which includes fields for title, description, category, condi
from django.db import models
from django.conf import settings
```

```
class Item(models.Model):
    CATEGORY_CHOICES = [
        ('Electronics', 'Electronics'),
        ('Books', 'Books'),
        ('Furniture', 'Furniture'),
        ('Sports', 'Sports'),
        ('Clothing', 'Clothing'),
    ]

    CONDITION_CHOICES = [
        ('Like New', 'Like New'),
        ('Good', 'Good'),
        ('Fair', 'Fair'),
        ('Poor', 'Poor'),
    ]

    STATUS_CHOICES = [
        ('Available', 'Available'),
        ('Sold', 'Sold'),
        ('Pending', 'Pending'),
    ]

    title = models.CharField(max_length=255)
    description = models.TextField(blank=True)
    category = models.CharField(max_length=20, choices=CATEGORY_CHOICES)
    condition = models.CharField(max_length=20, choices=CONDITION_CHOICES, default='Good')
    price = models.DecimalField(max_digits=10, decimal_places=2)
    old_price = models.DecimalField(max_digits=10, decimal_places=2, blank=True, null=True)
    seller = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.SET_NULL, null=True, blank=True, related_name='items')
    location = models.CharField(max_length=255, blank=True)
    image = models.ImageField(upload_to='marketplace_images/', blank=True, null=True)
    tags = models.JSONField(default=list, blank=True)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='Available')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-created_at']
```

```

# Serializers for the marketplace application, converting Item model instances to JSON format for API responses and validating input data for creating and updating marketplace items
from rest_framework import serializers
from django.utils import timezone
from .models import Item

class ItemSerializer(serializers.ModelSerializer):
    seller_name = serializers.CharField(source='seller.get_full_name', read_only=True)
    seller_email = serializers.EmailField(source='seller.email', read_only=True)
    posted = serializers.SerializerMethodField()
    image = serializers.ImageField(required=False, allow_null=True)

    class Meta:
        model = Item
        fields = ['id', 'title', 'description', 'category', 'condition', 'price', 'old_price', 'seller', 'seller_name', 'seller_email', 'location', 'image', 'tags', 'status']
        read_only_fields = ['id', 'seller', 'status', 'created_at', 'updated_at']

    def get_posted(self, obj):
        delta = timezone.now() - obj.created_at
        if delta.seconds < 3600:
            return f"{delta.seconds // 60} min ago"
        elif delta.days == 0:
            return f"{delta.seconds // 3600} hr ago"
        elif delta.days == 1:
            return "1 day ago"
        else:
            return f"{delta.days} days ago"

class ItemCreateSerializer(serializers.ModelSerializer):
    image = serializers.ImageField(required=False, allow_null=True)
    old_price = serializers.CharField(required=False, allow_null=True, allow_blank=True)
    tags = serializers.CharField(required=False, allow_blank=True)

    class Meta:
        model = Item
        fields = ['title', 'description', 'category', 'condition', 'price', 'old_price', 'location', 'image', 'tags']

    def to_internal_value(self, data):
        # Convert form data to proper types before validation
        ret = super().to_internal_value(data)

# URL configuration for the marketplace application, defining API endpoints for managing marketplace items
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from . import views

router = DefaultRouter()
router.register('', views.ItemViewSet)

urlpatterns = [
    path('', include(router.urls)),
]

# Admin configuration for the feedback application, registering the Feedback model with the Django admin interface to allow for easy management
from django.contrib import admin
from .models import Feedback

@admin.register(Feedback)
class FeedbackAdmin(admin.ModelAdmin):
    list_display = ['id', 'target_name', 'feedback_type', 'rating', 'is_anonymous', 'created_at']
    list_filter = ['feedback_type', 'rating', 'is_anonymous', 'created_at']
    search_fields = ['comment', 'target_name']
    readonly_fields = ['created_at', 'updated_at']

```

```

# Models for the feedback application, defining the Feedback model which includes fields for user (linked to the user model), feedback type, target
from django.db import models
from django.conf import settings

class Feedback(models.Model):
    FEEDBACK_TYPES = [
        ('course', 'Course/Academic'),
        ('teacher', 'Teacher/Staff'),
        ('hostel', 'Hostel/Facilities'),
        ('cafeteria', 'Cafeteria'),
        ('sports', 'Sports'),
        ('admin', 'Administrative'),
    ]

    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, null=True, blank=True)
    feedback_type = models.CharField(max_length=20, choices=FEEDBACK_TYPES)
    target_name = models.CharField(max_length=255) # Category or specific target
    rating = models.IntegerField(choices=[(i, i) for i in range(1, 6)]) # 1-5 stars
    comment = models.TextField()
    is_anonymous = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-created_at']

    def __str__(self):
        return f"Feedback by {self.user.username if self.user and not self.is_anonymous else 'Anonymous'} - {self.target_name}"

# URL configuration for the feedback application, defining API endpoints for managing feedback entries
> from django.urls import path, include...

router = DefaultRouter()
router.register('', views.FeedbackViewSet)

urlpatterns = [
    path('', include(router.urls)),
]

# D:\MajorProject\campVerse\campVerse_Backend\feedback - Contains emphasized items et which provides API endpoints for managing feedback entries. Tr
from rest_framework import viewsets, permissions
from rest_framework.decorators import action
from rest_framework.response import Response
from .models import Feedback
from .serializers import FeedbackSerializer, FeedbackCreateSerializer

class FeedbackViewSet(viewsets.ModelViewSet):
    queryset = Feedback.objects.all()
    serializer_class = FeedbackSerializer
    permission_classes = [permissions.AllowAny]

    def get_serializer_class(self):
        if self.action == 'create':
            return FeedbackCreateSerializer
        return FeedbackSerializer

    def perform_create(self, serializer):
        # If user is authenticated and not submitting anonymously, link to user
        if self.request.user.is_authenticated and not self.request.data.get('anonymous', False):
            serializer.save(user=self.request.user)
        else:
            serializer.save(user=None)

```

```

# User admin configuration for Django admin interface
from django.contrib import admin
from django.contrib.auth.admin import UserAdmin as BaseUserAdmin
from .models import User

@admin.register(User)
class UserAdmin(BaseUserAdmin):
    list_display = ['username', 'email', 'first_name', 'last_name', 'role', 'is_staff']
    list_filter = ['role', 'is_staff', 'is_superuser']
    fieldsets = BaseUserAdmin.fieldsets + (
        ('Additional Info', {'fields': ('role', 'phone', 'address', 'profile_image')}),
    )
    add_fieldsets = BaseUserAdmin.add_fieldsets + (
        ('Additional Info', {'fields': ('role', 'phone')}),
    )

# User model definition for the CampVerse application, extending Django's AbstractUser to include additional fields and ro
from django.db import models
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    STUDENT = 'student'
    TEACHER = 'teacher'
    ADMIN = 'admin'

    ROLE_CHOICES = [
        (STUDENT, 'Student'),
        (TEACHER, 'Teacher'),
        (ADMIN, 'Admin'),
    ]

    role = models.CharField(max_length=20, choices=ROLE_CHOICES, default=STUDENT)
    phone = models.CharField(max_length=20, blank=True)
    address = models.TextField(blank=True)
    profile_image = models.ImageField(upload_to='profile_images/', blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return self.username

```

```

from django.urls import path
from . import views

urlpatterns = [
    path('register/', views.RegisterView.as_view(), name='register'),
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),
    path('user/', views.user_view, name='user'),
    path('user/update/', views.update_user_view, name='update_user'),
]

```

```

# Views for the users application, defining API endpoints for user registration, login, logout, retrieving user data
from rest_framework import status, generics
from rest_framework.decorators import api_view, permission_classes, authentication_classes
from rest_framework.response import Response
from rest_framework.permissions import AllowAny, IsAuthenticated
from rest_framework.authentication import TokenAuthentication
from rest_framework.authtoken.models import Token
from django.contrib.auth import authenticate, get_user_model
from .serializers import UserSerializer, RegisterSerializer, LoginSerializer

User = get_user_model()

class RegisterView(generics.CreateAPIView):
    queryset = User.objects.all()
    serializer_class = RegisterSerializer
    permission_classes = [AllowAny]
    authentication_classes = []

    def create(self, request, *args, **kwargs):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        user = serializer.save()
        token, created = Token.objects.get_or_create(user=user)
        return Response({
            'token': token.key,
            'user': UserSerializer(user).data
        }, status=status.HTTP_201_CREATED)

@api_view(['POST'])
@permission_classes([AllowAny])
@authentication_classes([])
def login_view(request):
    serializer = LoginSerializer(data=request.data)
    if serializer.is_valid():
        username = serializer.validated_data['username']
        # Try to find user by username or email
        user = User.objects.filter(username=username).first()
        if not user:
            user = User.objects.filter(email=username).first()
        if user:
            user = authenticate(username=user.username, password=serializer.validated_data['password'])

#!/usr/bin/env python3
# manage.py - Django's command-line utility for administrative tasks. This script sets up the environment for the D
"""Django's command-line utility for administrative tasks."""
import os
import sys

def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'campverse.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()

```

```
# Django project dependencies listed in requirements.txt, specifying the required versions of Django,  
django>=4.2,<6.0  
djangorestframework>=3.14  
django-cors-headers>=4.0  
pillow>=10.0
```


Testing

Testing is a critical phase of the software development life cycle. It ensures that CampVerse performs correctly, securely, and efficiently under various conditions. The main objective of testing is to identify defects, verify that all modules work as expected, and ensure the system meets the project requirements.

CampVerse contains multiple modules (Lost & Found, Digital Notice Board, Goods Exchange, Hostel Management, Feedback System, etc.), so different testing techniques are used to check both functionality and performance.

1. Unit Testing

Unit testing involves testing each individual component or function of CampVerse separately.

Examples

- Testing login and signup validation
- Verifying database insert and fetch functions
- Checking image upload function for Lost & Found
- Testing form validation in Goods Exchange
- Verifying notice posting by faculty

Unit testing helps detect errors at an early stage.

2. Integration Testing

Integration testing checks the connection between modules and ensures that data flows properly across the system.

Examples

- Testing login → dashboard → notice board navigation
- Ensuring lost-item posts are correctly stored and displayed
- Verifying complaint submission → admin panel → response update
- Checking if goods posted by one user are visible to others

This ensures all modules work together smoothly.

3. System Testing

System testing evaluates the complete CampVerse system as a whole.

Areas Tested

- Full user journey from login to logout
- Loading time and response time
- Display compatibility on mobile and desktop
- Security features like invalid login detection
- Database consistency across all modules

System testing ensures CampVerse performs well in real-world conditions.

4. Functional Testing

Functional testing checks whether all features work according to requirements.

Examples

- Posting a notice
- Searching lost items
- Buying/selling items in Goods Exchange
- Submitting a complaint
- Allocating hostel rooms (if included)

This ensures the system meets expected functional specifications.

5. Non-Functional Testing

These tests examine performance, usability, and reliability.

a) Performance Testing

- Testing how fast notices load
- Checking response time under high traffic
- Measuring database execution speed

b) Usability Testing

- Checking if the interface is easy to use
- Ensuring navigation is simple for students and faculty
- Ensuring forms are user-friendly

c) Compatibility Testing

- Checking performance on different browsers
- Testing on mobile and desktop screens
- Checking responsiveness of UI

d) Security Testing

- Preventing unauthorized login
- Checking password protection and input validation
- Testing SQL injection prevention
- Ensuring safe file uploads

Non-functional testing ensures CampVerse is scalable, secure, and user-friendly.

6. Acceptance Testing

This final stage is performed by real users (faculty/admin/students) to verify if the system meets their expectations.

Includes

- Student checking Lost & Found and Goods Exchange
- Faculty posting a notice
- Admin reviewing complaints
- Users giving feedback about overall usability

If all users approve the system, CampVerse is marked as ready for deployment.

7. Regression Testing

Whenever updates or new features are added, regression testing is done to ensure nothing else breaks.

Example

- After adding a new notice filter option, checking if old notice features still work
- After modifying login, ensuring dashboard still loads correctly

Regression testing keeps the system stable during upgrades.

Conclusion

Testing ensures that CampVerse is reliable, secure, functional, and ready for use in a real campus environment. By performing unit, integration, system, functional, non-functional, and acceptance testing, the project becomes robust and error-free. Continuous testing also ensures long-term performance and smooth user experience.

Maintenance

Maintenance is an essential phase of the Software Development Life Cycle (SDLC). After CampVerse is deployed and students/faculty start using it, the system requires continuous monitoring, updates, and improvement. The goal of maintenance is to ensure that the platform remains **reliable, secure, user-friendly, and up-to-date** with the needs of the campus.

1. Corrective Maintenance

This type of maintenance focuses on fixing errors, bugs, and issues discovered after deployment.

- Fixing login/authentication issues
- Resolving broken links or incorrect data display
- Debugging database errors
- Fixing crashes, slow loading pages, or UI glitches
- Correcting notice upload or lost-and-found posting issues

Corrective maintenance ensures the system runs smoothly without disruptions.

2. Adaptive Maintenance

This is done when the system needs changes due to updates in the environment.

- Updating the system for new browser/mobile OS versions
- Adding support for new devices
- Changing features according to new college rules or academic structures
- Updating technology stack (e.g., upgrading PHP/Node.js versions)

Adaptive maintenance helps CampVerse stay compatible with new technologies and policies.

3. Perfective Maintenance

This type of maintenance is performed to improve existing features based on user feedback.

- Improving UI/UX for better usability
- Adding advanced search and filtering options
- Enhancing performance and speeding up loading time
- Upgrading the image upload quality for Lost & Found
- Improving dashboard layout for students and faculty

Perfective maintenance keeps CampVerse modern and user-friendly.

4. Preventive Maintenance

This maintenance helps detect and fix issues before they become serious problems.

- Regular data backup of notices, hostel records, and user data
- Monitoring server performance
- Checking for security vulnerabilities
- Removing outdated files or unused data
- Regularly cleaning and optimizing database tables

Preventive maintenance extends the life of the platform and avoids future failures.

5. Security Maintenance

Since CampVerse handles student data, notices, and complaints, strong security is necessary.

- Applying security patches
- Encrypting sensitive information
- Updating firewall and server security rules
- Monitoring unauthorized login attempts
- Regular penetration testing

This protects CampVerse from data breaches, hacking attempts, and misuse.

6. Database Maintenance

CampVerse relies heavily on database operations (notices, items, complaints, users).

- Regular indexing for faster queries
- Removing duplicate or unused records
- Creating automated backup schedules
- Updating database schema if features expand
- Ensuring high availability of databases

Good database maintenance improves speed and reliability.

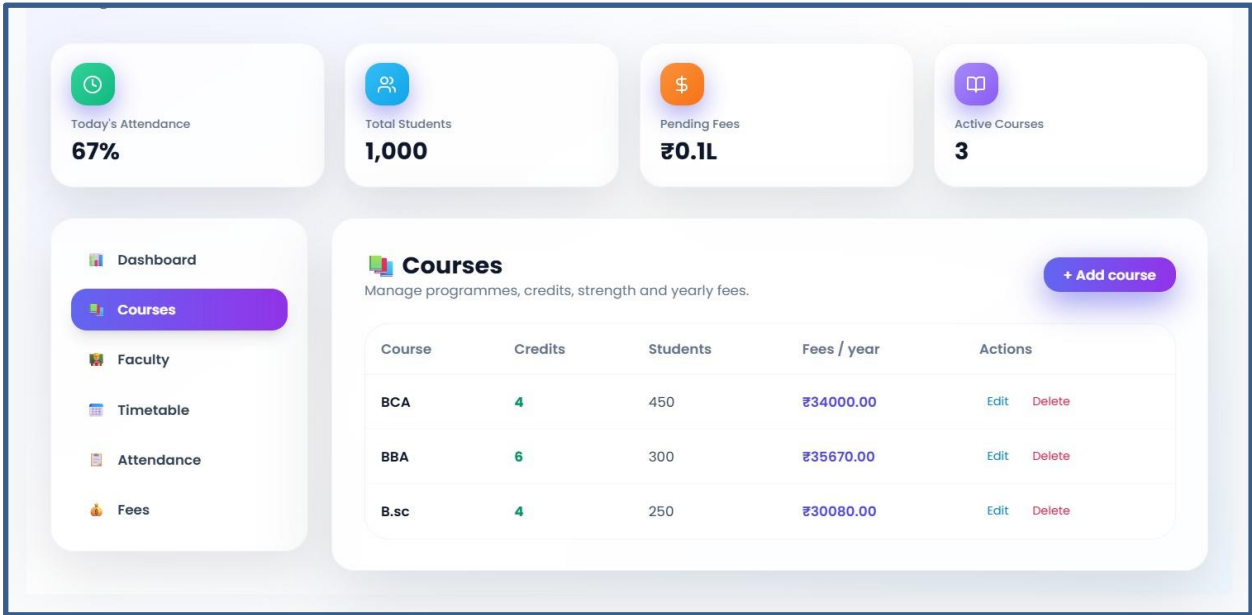
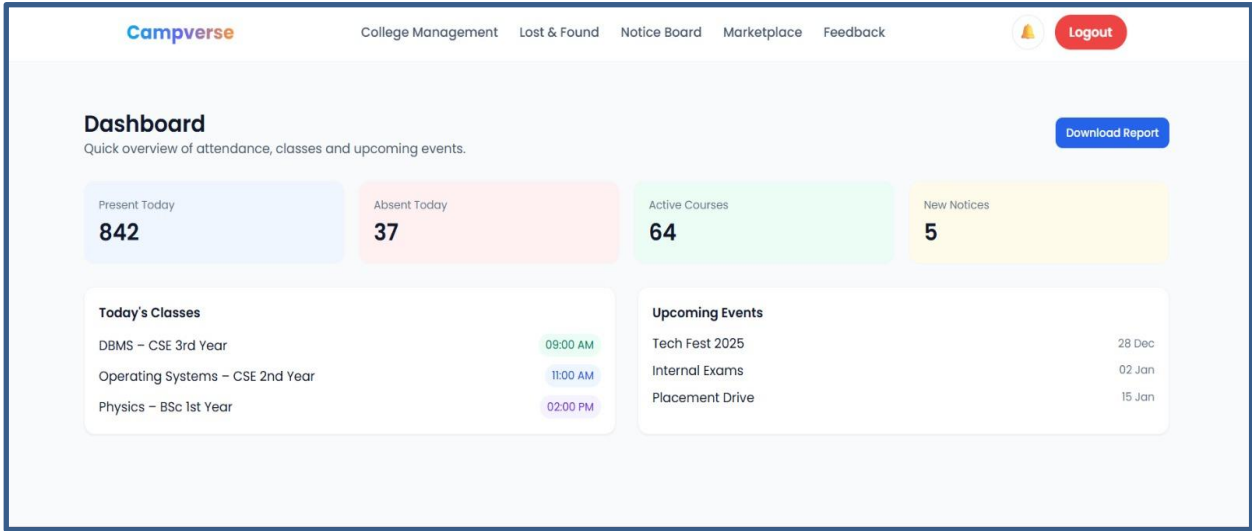
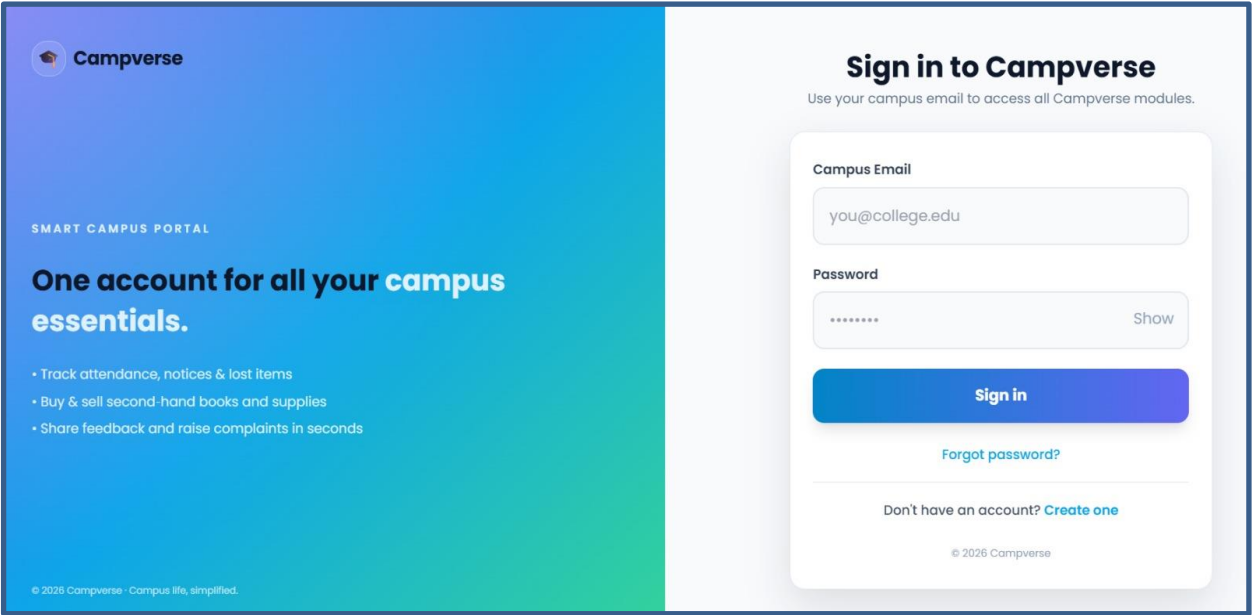
7. Feature Enhancement Maintenance

As the campus grows, new features may be added:

- Integrating AI assistant for campus queries
- Adding push notifications
- Introducing hostel fee payments
- QR-based class/hostel attendance
- More analytics and reports for admin

Enhancement maintenance ensures that CampVerse evolves with user needs.

Screen Shots



Campverse
College Management
Lost & Found
Notice Board
Marketplace
Feedback
Logout

CAMPVERSE MODULE

College Management

Central dashboard for courses, faculty, timetable, attendance and fee management.

Today's Attendance
67%

Total Students
450

Pending Fees
₹0.1L

Active Courses
1

Dashboard
Courses
Faculty
Timetable
Attendance

Dashboard overview

Today's Attendance
94%
↑ 2% from yesterday

Total Students
2,847
↑ 12 this month

Active Courses
28
Across 5 depts

Faculty
156
12 new this year

Attendance This Week

Fee Collection Status

Collected **₹18.5L (82%)**

Campverse
College Management
Lost & Found
Notice Board
Marketplace
Feedback
Logout

campus connected.

Total Listings
1

Lost Items
1

Found Items
0

+ Report Item

Create a new lost or found listing Instant

Item Title

Description

Location

Contact Email

Post Item

All

C++ Book
Lost

My C++ has lost last Monday.
Room : 204 2026-04-11

Contact
Delete

Campverse
College Management
Lost & Found
Notice Board
Marketplace
Feedback
Logout

CAMPUS UPDATES

Notice Board

Live digital notices with categories, timestamps and quick filters for your campus.

Mark All Read
+ New Notice

2 total
Latest: Just now
All
Academics
General
Hostel
Events

🚀 **Tech Club Recruitment Open!**
Academics

Last Date: 18 April Venue: Lab 3 Time: 2:00 PM Register now from the Events section!
Just now • Live • Pin

📁 **Internship Opportunity Alert**
Academics

A startup company is offering internships for: Web Development Data Analysis UI/UX Design Apply before 20 April in Placement Cell.
Just now • Live • Pin

Campverse
College Management
Lost & Found
Notice Board
Marketplace
Feedback

2
Logout

Student Feedback System

Your voice matters! Share your thoughts and suggestions to help us improve campus life and services.

1
Total Feedback

2.0
Avg Rating

0%
Implementation Rate

892
Active Participants

Submit Your Feedback

Help us improve by sharing your experience and suggestions.

Category

Academic Services
Library Services
Hostel Facilities
Cafeteria/Food
Sports Facilities
Administrative Services

Rating

★ ★ ★ ★ ★

Your Feedback

Share your thoughts, suggestions, or concerns...

☐ Submit anonymously

Submit Feedback

Recent Feedback

Academic Services
Acknowledged

"good"
★ ★ ★ ★ ★
34 min ago
Anonymous

Campus Marketplace

Sell an Item

Add item details to list it on Campus Marketplace.

Item title

e.g. Scientific Calculator, Bicycle

Item Image

Choose File No file chosen

Category

Electronics

Condition

Good

Price (₹)

Seller name

Your name

Location

e.g. Hostel C, Room 108

Tags (comma separated)

e.g. Laptop, Apple, 256GB

Cancel Post Item

Campverse
College Management
Lost & Found
Notice Board
Marketplace
Feedback

2
Logout

Campus Marketplace

Your campus OLX – Buy and sell second-hand items within your student community safely and easily.

Search products, books, electronics...

+ Sell Item

All Electronics Books Furniture Sports Clothing

No items found. Try another search.

Campverse

Your comprehensive campus platform connecting students, services and community. Making campus life easier, one click at a time.

Quick Links

Lost & Found
Notice Board
Marketplace
Feedback
College Management

Stay Connected

support@campverse.edu
+91 1234567890
Campus Administration Building

Limitations/Future Scope

Limitations of CampVerse

Although CampVerse provides a modern digital campus solution, it still has some limitations that may affect its performance or functionality. These limitations are common in developing projects and can be improved in future updates.

1. Limited Real-Time Communication

CampVerse does not include fully integrated real-time chat or instant messaging features. Students and faculty rely mainly on posts, notices, or forms instead of live communication.

2. Dependency on Internet Connection

As a web-based system, CampVerse requires a stable internet connection. In areas with poor connectivity, students may face difficulties accessing notices, lost & found, or goods exchange.

3. Limited Automation

Currently, many tasks such as hostel management, approval of complaints, and verification of goods are still semi-manual. Full automation is not implemented.

4. Security Constraints

Although basic security is provided, features like advanced encryption, multi-factor authentication, and biometric security are not included at this stage.

5. No Integration With College ERP

CampVerse runs as a standalone system. It is not connected with existing ERP systems like attendance, exam management, or academic records, which limits its overall usefulness.

6. Storage Limitations for Images/Files

Uploading images for Lost & Found or Goods Exchange depends on server storage. Heavy usage may cause storage issues if not managed properly.

7. Limited Personalization

The system does not yet support personalized dashboards based on user behaviour, preferences, or AI recommendations.

Future Scope of CampVerse

CampVerse has strong potential to expand and become a complete digital campus ecosystem. The following future improvements can enhance user experience, security, automation, and functionality.

1. Integration of Artificial Intelligence (AI)

- AI chatbot to answer student queries
- Smart recommendations for books, goods, or notices
- AI-based complaint sorting and priority handling

2. Mobile App Development

- Push notifications
- Offline access
- GPS for lost & found item locations

3. Complete ERP Integration

- Attendance system
- Examination and results module
- Library management
- Fee payment system
- Hostel biometric attendance

4. Advanced Security Enhancements

- Multi-factor authentication (OTP login)
- End-to-end encryption for sensitive data
- Role-based access control
- Secure file uploads and automatic threat detection

5. Automation of Hostel & Complaint System

- Auto-allotment of hostel rooms
- Automated complaint assignment to departments
- Real-time tracking of issue status
- Automatic reminders for pending tasks

6. Cloud-Based Scalability

- Storage capacity
- High-speed performance
- Auto-backups
- User load handling during peak hours

7. Integration of Digital Payments

- Hostel fee payment
- Event registration fees
- Fine payments
- Buying/selling goods with secure transactions

8. Analytics & Data Visualization

- Usage reports
- Student engagement charts
- Complaint trends
- Lost & Found success rates

9. QR & RFID Based Features

- QR code-based attendance
- RFID for hostel entry
- QR tags for identifying lost items

10. Multi-Campus Support

- Multiple colleges
- Universities
- Cross-campus communication

Conclusion

The limitations of CampVerse mainly arise from restricted automation, security, and integration. However, the future scope is vast—AI enhancements, ERP integration, automation, mobile app development, and cloud scalability can transform CampVerse into a powerful and complete digital campus ecosystem.

References/Bibliography

PROJECT: CampVerse (React+Django Based Web Application)

BOOKS:

- Learning React – Alex Banks & Eve Porcello
- Django for Beginners – William S. Vincent
- Full Stack React –Anthony Accomazzo

WEBSITES:

- <https://react.dev>
- <https://docs.djangoproject.com>
- <https://developer.mozilla.org>
- <https://www.w3schools.com>
- <https://stackoverflow.com>
- <https://www.geeksforgeeks.org>
- <https://tailwindcss.com>
- <https://axios-http.com>
- <https://vitejs.dev>
- <https://github.com>

TOOLS & TECHNOLOGIES:

- React JS
- Django
- Django REST Framework
- Tailwind CSS
- Axios
- SQLite / PostgreSQL
- Git & Github

SEARCH ENGINES:

Google, StackOverflow, ChatGPT, GitHub, MDN Documentation